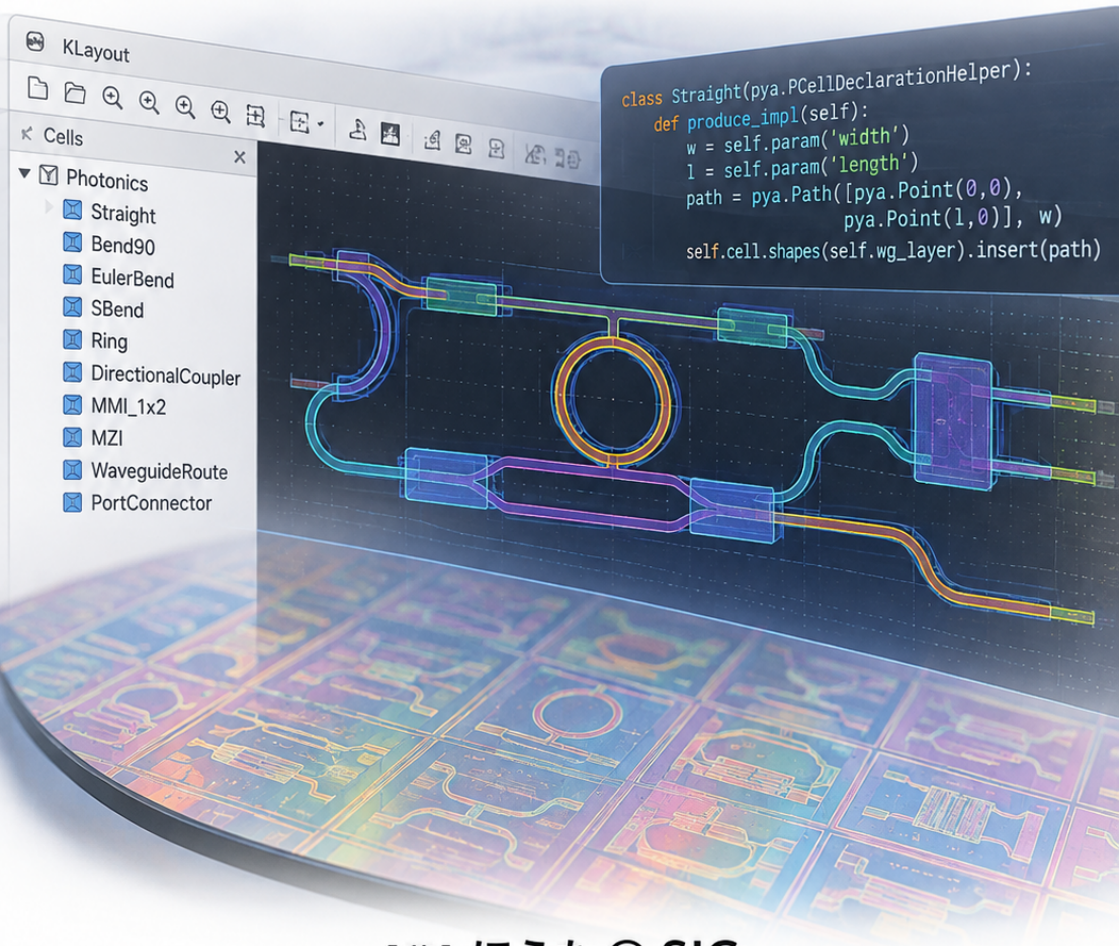
オープンソース PDK で遊ぼう 9

— KLayout でシリコンフォトニクス —



［著］ほうた @ SIG

技術書典 21 新刊
2026 年 11 月 21 日　ver 1.0

オープンソース PDK で遊ぼう 9

— KLayout でシリコンフォトニクス —

［著］ほうた@ SIG

技術書典 21 新刊
2026 年 11 月 21 日　 ver 1.0

■免責

本書は情報の提供のみを目的としています。
本書の内容を実行・適用・運用したことで何が起きようとも、それは実行・適用・運用した人自身の責任であり、著者や関係者はいかなる責任も負いません。

■商標

本書に登場するシステム名や製品名は、関係各社の商標または登録商標です。
また本書では、™、Ⓡ、Ⓒ などのマークは省略しています。

はじめに

本書は KLayout 用 Photonics PDK v4 の 10 種類の PCell について、ソースコードに基づき解説した技術書です。

本書は製造認定 PDK、光学シミュレーション結果、DRC 適合を保証するものではありません。

目次

はじめに i

第 1 章 はじめに 1
- 1.1 本書の目的 . . . 1
- 1.2 PDK とは . . . 1
- 1.3 対象環境 . . . 2
- 1.4 ライブラリの構成 . . . 2
- 1.5 幾何学的設計と光学的設計 . . . 3
- 1.6 本書の構成 . . . 3

第 2 章 KLayout と PCell の基本 5
- 2.1 PCell の概念 . . . 5
- 2.2 PCell の生成サイクル . . . 6
- 2.3 レイヤー . . . 6
- 2.4 DBU と座標変換 . . . 6
- 2.5 Path による導波路生成 . . . 7
- 2.6 Box による MMI 本体の生成 . . . 7
- 2.7 B é zier 曲線のサンプリング . . . 8
- 2.8 光ポートの表現 . . . 8
- 2.9 PCell インスタンスの変換 . . . 9
- 2.10 確認事項 . . . 9

第 3 章 PCell Library v4 のアーキテクチャ 11
- 3.1 ファイル構造 . . . 11
- 3.2 共通関数 . . . 11
- 3.3 Base クラス . . . 12
- 3.4 PCell クラスの登録 . . . 12
- 3.5 全 PCell の初期パラメータ . . . 13
- 3.6 生成処理の共通パターン . . . 13
- 3.7 設計上の特徴と制約 . . . 14
  - ♣ 光ポートは Path + Text . . . 14
  - ♣ 曲線は離散点列 . . . 14

♣ EulerBend の radius の解釈 14
♣ WaveguideRoute の radius の解釈 14
♣ MZI の対称性 15
♣ PortConnector の役割 15
3.8 Photonics Router との役割分担 15
3.9 将来の改善候補 15
3.10 まとめ 16

第 4 章 Straight — 直線導波路 17
4.1 概要と用途 17
4.2 構造と座標 17
♣ 初期値での形状 18
4.3 パラメータ 18
4.4 Python 実装 18
♣ 処理の流れ 19
4.5 DBU と整数座標 19
4.6 Optical Port の定義 20
4.7 導波路長と光学位相 21
4.8 KLayout での操作例 21
4.9 設計・検証上の注意 21
4.10 Photonics Router v6.x との連携 22
4.11 v5 への改善候補 23
4.12 まとめ 23
♣ PCell 構造図（v4 初期パラメータ） 23

第 5 章 Bend90 — 90°円弧導波路 25
5.1 概要と用途 25
5.2 基本構造と座標系 25
5.3 PCell パラメータ 26
5.4 Python による形状生成（参考実装） 26
5.5 円弧長の計算 27
5.6 分割数と円弧近似誤差 28
5.7 曲率と曲げ損失 28
5.8 光ポートの定義 29
5.9 KLayout での確認手順 29
5.10 Photonics Router v6.x との連携 29

5.11 v5 への改善案 . . . 30
5.12 まとめ . . . 30
♣ PCell 構造図（v4 初期パラメータ） . . . 30

第 6 章 EulerBend — Euler 型曲げ導波路 33
6.1 概要と用途 . . . 33
6.2 Bend90 との比較 . . . 34
6.3 v4 のパラメータ . . . 34
6.4 Euler 曲線の数学的原理 . . . 35
♣ Fresnel 積分 . . . 35
6.5 90°曲げを構成する方法 . . . 35
6.6 v4 のフットプリントと曲率半径 . . . 36
6.7 数値生成の参考コード . . . 36
6.8 離散化と DBU . . . 37
6.9 導波路長の計算 . . . 38
6.10 光ポートの考え方 . . . 38
6.11 KLayout での確認手順 . . . 39
6.12 Photonics Router v6.x との連携 . . . 39
6.13 PCell Library v5 への改善案 . . . 39
6.14 まとめ . . . 39
♣ PCell 構造図（v4 初期パラメータ） . . . 40

第 7 章 SBend — S 字導波路 41
7.1 概要 . . . 41
7.2 構造と座標 . . . 41
7.3 パラメータ . . . 42
7.4 3 次 B é zier 曲線 . . . 42
♣ 両端の接線 . . . 43
7.5 v4 の Python 実装 . . . 43
7.6 Optical Port . . . 44
7.7 導波路長 . . . 44
7.8 曲率と光学設計 . . . 45
7.9 離散化・DBU・入力値の注意 . . . 45
7.10 KLayout での確認手順 . . . 45
7.11 Photonics Router v6.x との連携 . . . 46
7.12 PCell Library v5 への改善案（未実装） . . . 46

7.13 まとめ 46
♣ PCell 構造図（v4 初期パラメータ） 46

第 8 章 Ring — リング共振器 49
8.1 概要 49
8.2 構造と座標系 49
8.3 PCell パラメータ 50
8.4 Python 実装の構造 50
8.5 リング長と共振条件 51
8.6 FSR（自由スペクトル間隔） 52
8.7 ギャップと結合 52
8.8 円周の離散化と誤差 53
8.9 Optical Port の定義 53
8.10 KLayout での確認手順 53
8.11 Photonics Router v6.x との連携 54
8.12 v5 への改善提案 54
8.13 まとめ 54
♣ PCell 構造図（v4 初期パラメータ） 55

第 9 章 DirectionalCoupler — 方向性結合器 57
9.1 概要 57
9.2 構造と座標系 57
9.3 PCell パラメータ 58
9.4 Python ソースコードの解説 59
9.5 3 次 B é zier アクセス導波路 60
9.6 Optical Port の定義 60
9.7 光結合の理論 61
9.8 導波路長と光学位相 62
9.9 離散化と DBU 62
9.10 KLayout での確認手順 62
9.11 Photonics Router v6.x との連携 63
9.12 v4 の制約と v5 への改善案 63
9.13 まとめ 64
♣ PCell 構造図（v4 初期パラメータ） 64

第 10 章 MMI_1x2 — 多モード干渉型 1 × 2 光分岐器 65

10.1 概要と用途 . . . 65
10.2 構造と座標系 . . . 65
10.3 PCell パラメータ . . . 66
10.4 Python 実装の解説 . . . 66
10.5 Optical Port の仕様 . . . 68
10.6 MMI の光学原理（設計背景） . . . 68
10.7 寸法条件と幾何学的な注意点 . . . 69
10.8 KLayout での確認手順 . . . 69
10.9 Photonics Router v6.x との連携 . . . 70
10.10 v5 への改善案（現行 v4 には未実装） . . . 70
10.11 まとめ . . . 70
♣ PCell 構造図（v4 初期パラメータ） . . . 71

第 11 章 MZI — マッハ・ツェンダー干渉計 73
11.1 概要 . . . 73
11.2 幾何学的構造 . . . 73
11.3 PCell パラメータ . . . 74
11.4 座標の計算 . . . 74
11.5 Python 実装の読み方 . . . 75
11.6 S 字遷移の数学的性質 . . . 76
11.7 Optical Port の仕様 . . . 76
11.8 光学干渉の原理 . . . 76
11.9 v4 の MZI が対称であることの意味 . . . 77
11.10 KLayout での確認手順 . . . 77
11.11 Photonics Router v6.x との連携 . . . 78
11.12 v5 に向けた改善案（未実装） . . . 78
11.13 まとめ . . . 78
♣ PCell 構造図（v4 初期パラメータ） . . . 79

第 12 章 WaveguideRoute — 導波路ルーティング 81
12.1 概要 . . . 81
12.2 幾何学的な考え方 . . . 81
12.3 PCell パラメータ . . . 82
12.4 3 次 Bé zier 曲線 . . . 83
12.5 曲率と最小曲率半径 . . . 83
12.6 導波路長 . . . 83

12.7 v4 の Python 実装（ソース照合済み） . . . 84
12.8 Optical Port（ソース照合済み） . . . 84
12.9 KLayout での確認手順 . . . 85
12.10 Photonics Router v6.x との関係 . . . 85
12.11 PCell Library v5 への改善提案 . . . 85
12.12 まとめ . . . 86
♣ PCell 構造図（v4 初期パラメータ） . . . 86

第 13 章 PortConnector — 光ポート接続・位置合わせ 87
13.1 概要 . . . 87
13.2 形状と座標 . . . 87
13.3 PCell パラメータ . . . 88
13.4 v4 の実装 . . . 88
♣ 処理の流れ . . . 89
13.5 Optical Port の仕様 . . . 89
13.6 Straight との比較 . . . 90
13.7 導波路長と光学位相 . . . 90
13.8 KLayout での確認手順 . . . 90
13.9 Photonics Router v6.x との連携 . . . 91
13.10 設計上の注意点 . . . 91
13.11 v5 への改善案（未実装） . . . 92
13.12 まとめ . . . 92
♣ PCell 構造図（v4 初期パラメータ） . . . 92

第 14 章 Optical Port の共通仕様 93
14.1 Optical Port とは . . . 93
14.2 v4 のレイヤー表現 . . . 94
14.3 ローカル座標と内向き方向 . . . 94
14.4 PCell インスタンスの回転と反転 . . . 95
♣ 数値例 . . . 95
14.5 10 種類の PCell とポート数 . . . 96
14.6 add_pin() の役割 . . . 96
♣ DBU 変換の例 . . . 97
14.7 ポート接続の成立条件 . . . 97
♣ 位置が一致していない場合 . . . 97
14.8 Router v6.x との連携 . . . 97

14.9 実装検証チェックリスト . . . 98
♣ 推奨テストケース . . . 98
14.10 PCell Library v5 への改善案 . . . 99
14.11 まとめ . . . 99

第 15 章 Photonics Router v6.x との連携 101
15.1 本章の目的 . . . 101
15.2 システム全体の構成 . . . 102
15.3 Optical Port 抽出 . . . 103
♣ 入力情報 . . . 103
♣ 階層座標への変換 . . . 103
♣ 内向き／外向きの規約 . . . 103
15.4 ポート間接続の成立条件 . . . 104
15.5 導波路ルーティング方式 . . . 104
♣ WaveguideRoute PCell との違い . . . 104
♣ 曲線の候補 . . . 104
15.6 曲率制約と幾何学的検証 . . . 105
15.7 導波路長の計算 . . . 105
15.8 光ネットリスト抽出 . . . 106
♣ 推奨する識別モデル . . . 106
15.9 KLayout での検証シナリオ . . . 107
15.10 推奨する共通データモデル（v5 以降の提案） . . . 107
15.11 実装・公開前の確認事項 . . . 108
15.12 まとめ . . . 108

第 16 章 PCell Library v5 への発展 — 共通化・検証・Router 統合 109
16.1 開発方針 . . . 109
16.2 提案するソフトウェア構成 . . . 110
16.3 Optical Port API の統一 . . . 111
♣ 方向の規約 . . . 111
♣ 一意なポート識別 . . . 111
16.4 共通幾何学エンジン . . . 112
16.5 パラメータと設計ルールの検証 . . . 112
16.6 10 種類の PCell 別改善候補 . . . 113
16.7 Photonics Router との共通インターフェース . . . 113
16.8 テスト自動化と CI . . . 114
♣ ユニットテスト . . . 114

♣ 回帰テスト 114
♣ 統合テスト 114
16.9 段階的な開発ロードマップ 114
16.10 v4 から v5 への互換性 115
16.11 公開前チェックリスト 115
16.12 まとめ 116

付録 A KLayout 実機検証記録 117

第 1 章
はじめに

1.1 本書の目的

シリコンフォトニクスのレイアウトでは、導波路、曲げ、分岐器、共振器、干渉計などの構造を繰り返し使用します。PCell（Parameterized Cell）を利用すると、これらの構造を**パラメータ付きの再利用可能な部品**として定義できます。

本書の目的は、KLayout 上で動作する **Photonics PDK v4** の 10 種類の PCell について、次の点を理解できるようにすることです。

1. 各 PCell が生成する形状とその用途
2. パラメータが形状に与える影響
3. Python（`pya`）による生成処理
4. 光ポートの配置と方向の規約
5. 幾何学的妥当性と光学的性能評価の違い
6. Photonics Router との連携上の課題

1.2 PDK とは

PDK（Process Design Kit）は一般に、特定の製造プロセスで設計するためのレイヤー定義、設計ルール、デバイスライブラリ、モデルなどを含む設計資産の集合を指します。

本プロジェクトの **Photonics PDK v4** は、Python で光導波路形状を生成する PCell ライブラリを中心とした実装です。名称に「PDK」を含みますが、**製造ファウ**

ンドリ認定の完全な PDK や、検証済みの光学コンパクトモデル一式を提供していることを意味しません。製造利用には、対象プロセスの DRC・レイヤーマップ・光学モデルとの照合が必要です。

1.3　対象環境

- レイアウトエディタ：KLayout
- マクロ形式：`.lym`（KLayout マクロ）
- スクリプト：Python（`pya` API）
- PCell ライブラリ登録名：`Photonics`
- 実装ファイル：`pymacros/photonics_pdk_v4.lym`

v4 のマクロは XML 形式で、`<interpreter>python</interpreter>`、`<autorun>true</autorun>`が設定されています。コード中のコメントには KLayout 0.30.x を対象とする記載がありますが、**各 OS・KLayout バージョンでの動作確認結果を示すものではありません**。

1.4　ライブラリの構成

▼表 1.1: パラメータ・仕様一覧

分類	PCell	代表的な役割
基本導波路	Straight	直線
基本導波路	Bend90	90°曲げ
基本導波路	EulerBend	曲率を変化させる 90°曲げ
基本導波路	SBend	横方向のオフセット
光デバイス	Ring	バス導波路とリング
光デバイス	DirectionalCoupler	近接導波路による結合
光デバイス	MMI_1x2	1 × 2 分岐構造
光デバイス	MZI	2 アーム干渉構造
ルーティング補助	WaveguideRoute	2 点間の曲線
ルーティング補助	PortConnector	短い直線と 2 つのポート

1.5　幾何学的設計と光学的設計

PCell のパラメータは、基本的には**レイアウト図形**を制御します。例えば、MMI の幅・長さを設定しても、それだけで 50:50 分岐や低損失を保証するわけではありません。同様に、B é zier 曲線が滑らかに見えても、所定の最小曲率半径を満たすとは限りません。

本書では以下を明確に区別します。

- **実装仕様**：v4 のコードが生成する図形・座標・ポート
- **理論**：光の伝搬・干渉・結合に関する設計原理
- **検証事項**：DRC、電磁界シミュレーション、製造ばらつき評価
- **改善提案**：将来版で追加を検討する機能

1.6　本書の構成

第 2 章では KLayout の PCell と Python API の基本を説明し、第 3 章では v4 ライブラリの内部構成を確認します。第 4〜13 章で 10 種類の PCell を個別に解説し、応用編でポート仕様と Photonics Router との連携を扱います。

参照:　v4 マクロ（GitHub）　(https://github.com/TAKE-HooJoo/klayout-photonics-pdk/blob/main/pymacros/photonics_pdk_v4.lym)

第 2 章

KLayout と PCell の基本

2.1 PCell の概念

PCell は、同じ種類の図形をパラメータから生成するセルです。例えば Straight では、`length` と `width` を変えることで、異なる長さ・幅の導波路を同じ PCell 定義から生成できます。

KLayout の Python PCell は、`pya.PCellDeclarationHelper` を継承して作成できます。

▼ python

```
class Straight(pya.PCellDeclarationHelper):
    def __init__(self):
        super(Straight, self).__init__()
        self.param("length", self.TypeDouble, "Length (um)", de›
›fault=100.0)
        self.param("width", self.TypeDouble, "Width (um)", defa›
›ult=0.5)

    def produce_impl(self):
        # レイアウト図形の生成処理
        pass
```

上記は API の最小構成を説明する**簡略例**です。実際の v4 では、共通の `Base` クラスを継承し、導波路レイヤーと Pin レイヤーも登録します。

2.2　PCell の生成サイクル

1. PCell クラスを宣言する。
2. `self.param()` でパラメータを定義する。
3. `produce_impl()` に形状生成処理を書く。
4. `layout.register_pcell()` でライブラリに登録する。
5. KLayout で PCell インスタンスを配置し、パラメータを指定する。

`produce_impl()` では、`self.layout` と `self.cell` を通して、図形を配置する対象レイアウト・セルにアクセスします。

2.3　レイヤー

v4 の標準レイヤー定義は次のとおりです。

▼ python

```
WG_LAYER  = pya.LayerInfo(1, 0)
PIN_LAYER = pya.LayerInfo(2, 0)
```

▼ 表 2.1: パラメータ・仕様一覧

レイヤー	用途
1/0	光導波路の形状
2/0	光ポートの Path と Text

これは v4 の**ライブラリ内での初期設定**です。特定ファウンドリの製造レイヤー番号であることを保証しません。

2.4　DBU と座標変換

KLayout の `pya.Point` や `pya.Path` では整数の DBU 座標を使用します。v4 には、μ m 単位の値を DBU へ変換する関数があります。

▼ python

```
def q(v, dbu):
    return int(round(v / dbu))
```

例えば DBU が 0.001 μ m の場合、10 μ m は 10,000 DBU です。

$$N_{\mathrm{DBU}} = \mathrm{round}\left(\frac{x_{\mu m}}{\mathrm{DBU}}\right)$$

注意： DBU への丸めにより、微小な座標誤差が生じます。曲線の分割点数や細い導波路の幅は、使用する DBU に対して適切に設定する必要があります。

2.5　Path による導波路生成

v4 では、導波路の中心線を点列で定義し、`pya.Path` に変換します。

▼ python

```
def path_um(points, width, dbu):
    return pya.Path(
        [pya.Point(q(x, dbu), q(y, dbu)) for x, y in points],
        q(width, dbu)
    )
```

例えば、

▼ python

```
path_um([(0, 0), (20, 0)], 0.5, layout.dbu)
```

は、中心線が (0,0) から (20,0) までの直線で、幅 0.5 μ m の Path を作ります。`Path` の幾何学的な端部形状は KLayout の Path 仕様に従います。

2.6　Box による MMI 本体の生成

MMI のような矩形領域は `pya.Box` で生成します。

▼ python

```
pya.Box(
    q(xmin, layout.dbu), q(ymin, layout.dbu),
    q(xmax, layout.dbu), q(ymax, layout.dbu)
)
```

Box は長方形の幾何学形状であり、光学的な自己結像条件や結合効率を計算する機能ではありません。

2.7　B é zier 曲線のサンプリング

v4 では 3 次 B é zier 曲線を点列に離散化する bezier() 関数を使用します。

$$B(t) = (1-t)^3 P_0 + 3(1-t)^2 t P_1 + 3(1-t) t^2 P_2 + t^3 P_3, \quad 0 \le t \le 1$$

制御点 P0〜P3 によって端点と接線方向を調整できます。生成した点列は path_um() で導波路図形に変換します。

注意： 曲線の端点で接線が一致しても、曲率が連続するとは限りません。また、制御点の距離は必ずしも最小曲率半径に一致しません。

2.8　光ポートの表現

v4 では、add_pin() が Pin レイヤー上に短い Path とポート名 Text を生成します。

▼ python

```
add_pin(cell, layout, pin_layer, "opt1", x, y, inward_angle, wi›
›dth)
```

inward_angle は**ポートからセル内部へ向かう方向**です。Router で外向き方向を使う場合には、次の変換が必要です。

$$\theta_{\text{out}} = (\theta_{\text{in}} + 180^\circ) \bmod 360^\circ$$

v4 の光ポートは、位置・方向・名前をレイアウト図形として表現したものです。独立したネットリスト接続オブジェクトが自動生成されるわけではありません。

2.9 PCell インスタンスの変換

PCell インスタンスは、配置時に平行移動、回転、反転できます。Router が接続先を判定するときは、**PCell ローカル座標のポート情報を、インスタンス変換後の座標へ変換**する必要があります。

2.10 確認事項

KLayout で PCell を使用する際には、以下を確認します。

- マクロが読み込まれ、`Photonics` ライブラリが表示されること
- パラメータ変更で形状が再生成されること
- 導波路レイヤーと Pin レイヤーが意図どおりであること
- ポートの位置と方向が図形の端点に一致すること
- 生成された形状が対象プロセスの設計ルールに適合すること

参照: v4 マクロ（GitHub） (https://github.com/TAKE-HooJoo/klayout-photonics-pdk/blob/main/pymacros/photonics_pdk_v4.lym)

第 3 章

PCell Library v4 のアーキテクチャ

3.1 ファイル構造

v4 の実装は、KLayout マクロファイル `photonics_pdk_v4.lym` にまとめられています。ファイルは XML 形式で、`<text>`要素に Python コードを格納しています。

▼ text

```
photonics_pdk_v4.lym
 ├── XMLマクロ設定（Python / autorun）
 └── Pythonコード
     ├── レイヤー定義
     ├── 共通幾何学関数（q, path_um, bezier）
     ├── 光ポート生成関数（add_pin）
     ├── 共通Baseクラス
     ├── 10種類のPCellクラス
     └── PhotonicsLibrary（登録処理）
```

3.2 共通関数

これらの関数は複数の PCell から呼び出され、図形生成方法を共通化しています。

▼表 3.1: パラメータ・仕様一覧

関数	役割
`q(v, dbu)`	μ m 値を整数 DBU に変換
`path_um(points, width, dbu)`	点列から Path を生成
`bezier(p0, p1, p2, p3, n)`	3 次 B é zier 曲線を点列化
`add_pin(...)`	Pin レイヤーに Path と Text を生成

3.3 Base クラス

v4 の Base クラスは、各 PCell に共通する 2 つのレイヤーパラメータを定義します。

▼python

```
class Base(pya.PCellDeclarationHelper):
    def layers(self):
        self.param("layer", self.TypeLayer,
                   "Waveguide layer", default=WG_LAYER)
        self.param("pin_layer", self.TypeLayer,
                   "Optical pin layer", default=PIN_LAYER)
```

各 PCell は `Base` を継承し、コンストラクタで `self.layers()` を呼び出します。

3.4 PCell クラスの登録

ライブラリ登録処理では、次の 10 種類の名前が使用されます。

▼python

```
class PhotonicsLibrary(pya.Library):
    def __init__(self):
        self.description = "Simple Silicon Photonics PCell Libr›
›ary v4"
        z = self.layout()
        for name, pc in [
            ("Straight", Straight()),
            ("Bend90", Bend90()),
            ("EulerBend", EulerBend()),
            ("SBend", SBend()),
```

```
            ("Ring", Ring()),
            ("DirectionalCoupler", DirectionalCoupler()),
            ("MMI_1x2", MMI1x2()),
            ("MZI", MZI()),
            ("WaveguideRoute", WaveguideRoute()),
            ("PortConnector", PortConnector())
        ]:
            z.register_pcell(name, pc)
        self.register("Photonics")

PhotonicsLibrary()
```

上記は v4 の登録部分を**整形して掲載**したものです。ライブラリ名は `Photonics` で、`MMI_1x2` の内部 Python クラス名は `MMI1x2` です。

3.5　全 PCell の初期パラメータ

▼表 3.2: パラメータ・仕様一覧

PCell	主要パラメータ（初期値、μ m。ただし points は無次元）
Straight	length=100, width=0.5
Bend90	radius=10, width=0.5, points=64
EulerBend	radius=10, width=0.5, points=96
SBend	length=20, offset=5, width=0.5, points=64
Ring	radius=10, width=0.5, gap=0.2, bus_length=40, points=128
DirectionalCoupler	coupling_length=20, access_length=15, gap=0.2, port_spacing=5, width=0
MMI_1x2	wg_width=0.5, mmi_width=4, mmi_length=20, port_spacing=2, access_l
MZI	mmi_width=4, mmi_length=20, access_length=5, arm_length=80, arm_sp
WaveguideRoute	dx=100, dy=20, width=0.5, radius=10, points=24
PortConnector	length=10, width=0.5

共通パラメータ: 全 PCell で `layer=1/0`、`pin_layer=2/0` を指定できます。

3.6　生成処理の共通パターン

ほとんどの PCell は、次の処理を実行します。

▼ text

```
PCellパラメータ
      ↓
中心線または矩形の座標計算
      ↓
DBU変換
      ↓
WGレイヤーへ図形を挿入
      ↓
Pinレイヤーへ光ポートを挿入
      ↓
PCellインスタンスとして配置
```

この構造は理解しやすく、個別の PCell を拡張しやすい利点があります。

3.7　設計上の特徴と制約

♣ 光ポートは Path ＋ Text

各 PCell は `add_pin()` を使用します。ポート名は `opt1`、`opt2` などです。ただし、ポートメタデータが別の構造化データとして保持されるわけではありません。

♣ 曲線は離散点列

円弧・Bézier 曲線・Euler 型曲線は点列として生成され、`pya.Path` へ変換されます。分割数と DBU が形状精度に影響します。

♣ EulerBend の radius の解釈

v4 の `EulerBend` では、生成した曲線を X/Y 方向にスケーリングして終点を `(radius, radius)` に合わせています。したがって、パラメータ名の `radius` は**フットプリント寸法の指定**であり、厳密な最小曲率半径を保証する値ではありません。

♣ WaveguideRoute の radius の解釈

v4 の `WaveguideRoute` では、`radius` を Bézier 曲線の制御距離の計算に使用します。**実際の曲線の最小曲率半径が `radius` になるわけではありません。**

♣ MZI の対称性

MZI の下側アームは、上側アームの Y 座標を反転して生成しています。したがって、中心線の幾何学的長さは上下で等しく、v4 には意図的な光路長差を与えるパラメータはありません。

♣ PortConnector の役割

PortConnector は、短い直線導波路と 2 つの光ポートを生成します。外部 PCell のポートを探索する自動接続アルゴリズムは実装されていません。

3.8　Photonics Router との役割分担

▼表 3.3: パラメータ・仕様一覧

PCell Library v4	Photonics Router（連携対象）
部品内部の図形を生成	配置済み部品のポートを検出
パラメータから形状を生成	ポート間の経路を計算
Pin レイヤーにポートを記述	回転・反転後のポート座標を扱う
個々の光デバイスを表現	部品間の接続関係を抽出

Router の具体的な機能・実装状態は、対象バージョンの Router ソースコードを別途確認して記載します。

3.9　将来の改善候補

1. ポート座標・角度・幅・名前を共通仕様で管理する。
2. 曲線の最小曲率半径を数値的に検証する。
3. 導波路中心線長を統一的に計算する。
4. PCell のパラメータ妥当性チェックを追加する。
5. 光学性能の推定値と、電磁界シミュレーションによる検証結果を区別して記録する。
6. 参照コミット ID と KLayout バージョンを解説書に固定する。

3.10　まとめ

Photonics PDK v4 は、共通の幾何学関数と光ポート生成関数を利用する 10 種類の PCell から構成されています。現状の中心機能は**パラメトリックなレイアウト生成**です。今後、Photonics Router との統合や光学検証を進めるには、ポート情報・曲率検証・導波路長の共通化が重要です。

参照:　v4 マクロ（GitHub）　(https://github.com/TAKE-HooJoo/klayout-photonics-pdk/blob/main/pymacros/photonics_pdk_v4.lym)

第 4 章

Straight — 直線導波路

実装基準：`pymacros/photonics_pdk_v4.lym`（https://github.com/TAKE-HooJoo/klayout-photonics-pdk/blob/main/pymacros/photonics_pdk_v4.lym）の `Straight` クラス（ソース 39 – 47 行付近）。以下のソースは可読性のため改行・整形しています。`main` ブランチは更新されるため、正式公開時は対象コミットを固定してください。

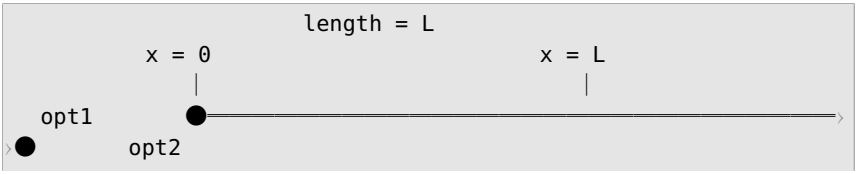
4.1 概要と用途

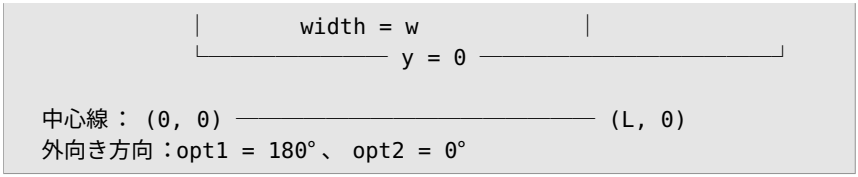
`Straight` は、幅が一定の直線光導波路を生成する基本 PCell です。光回路内でデバイス間を接続するほか、光路長の調整、レイアウト検証、光ポート認識の動作確認に利用できます。

本 PCell は**導波路の幾何学形状**を生成します。伝搬損失・実効屈折率・群屈折率などを自動計算するものではありません。

4.2 構造と座標

▼ text





導波路の中心線は `[(0, 0), (length, 0)]` で定義されます。幅は中心線に対して対称に与えられます。

♣ 初期値での形状

- 中心線長：**100.0 μ m**
- 導波路幅：**0.5 μ m**
- 始点：**(0, 0) μ m**
- 終点：**(100, 0) μ m**

図の端部形状は `pya.Path` の表現・変換に依存します。ここでの「長さ」は**中心線長**を指します。

4.3　パラメータ

▼表 4.1: パラメータ・仕様一覧

パラメータ	型	初期値	単位	説明
`length`	`TypeDouble`	100.0	μ m	導波路中心線長
`width`	`TypeDouble`	0.5	μ m	導波路幅
`layer`	`TypeLayer`	1/0	—	導波路を描画するレイヤー
`pin_layer`	`TypeLayer`	2/0	—	ポートを描画するレイヤー

`layer` と `pin_layer` は `Base.layers()` から継承します。v4 のコードには `length > 0` や `width > 0` の明示的な入力検証がないため、運用時には正の値を指定してください。

4.4　Python 実装

v4 の `Straight` クラスを読みやすく整形すると次のようになります。

▼ python

```
class Straight(Base):
    def __init__(self):
        super(Straight, self).__init__()
        self.param("length", self.TypeDouble, "Length (um)", de›
›fault=100.0)
        self.param("width", self.TypeDouble, "Width (um)", defa›
›ult=0.5)
        self.layers()

    def produce_impl(self):
        l, c = self.layout, self.cell
        wg = l.layer(self.layer)
        pin = l.layer(self.pin_layer)

        c.shapes(wg).insert(
            path_um([(0, 0), (self.length, 0)], self.width, l.d›
›bu)
        )

        add_pin(c, l, pin, "opt1", 0, 0, 0, self.width)
        add_pin(c, l, pin, "opt2", self.length, 0, 180, self.wi›
›dth)
```

♣ 処理の流れ

1. `self.layout` と `self.cell` からレイアウトと PCell 生成先セルを取得します。
2. `l.layer(...)` で導波路レイヤーと Pin レイヤーの内部インデックスを取得します。
3. `path_um(...)` でμ m 単位の中心線と幅から `pya.Path` を生成します。
4. `c.shapes(wg).insert(...)` で導波路を描画します。
5. `add_pin(...)` を 2 回呼び出して、両端に名前付き光ポートを配置します。

4.5 DBU と整数座標

共通関数は以下のとおりです。

▼ python

```
def q(v, dbu):
    return int(round(v / dbu))

def path_um(points, width, dbu):
    return pya.Path(
        [pya.Point(q(x, dbu), q(y, dbu)) for x, y in points],
        q(width, dbu)
    )
```

`q()` はμ m 単位の数値を DBU 整数座標へ変換します。例えば **DBU = 0.001 μ m** の場合、

$$100.0\,\mu\mathrm{m} \longrightarrow 100000 \text{ DBU}$$

$$0.5\,\mu\mathrm{m} \longrightarrow 500 \text{ DBU}$$

となります。DBU はレイアウトごとに異なるため、`100000` や `500` をコードに固定しないことが重要です。寸法は DBU に丸められるため、設定値と実際の GDS 寸法には最大で概ね半 DBU 程度の量子化誤差が生じ得ます。

4.6 Optical Port の定義

共通関数 `add_pin()` は、Pin レイヤーに短い Path とポート名 Text を挿入します。引数 `inward_angle` は、**ポートから PCell 内部へ向かう角度**です。

▼ 表 4.2: パラメータ・仕様一覧

ポート	中心位置（μ m）	内向き角度	外向き角度	幅
opt1	(0, 0)	0°	180°	width
opt2	(length, 0)	180°	0°	width

外向き角度への変換は次のとおりです。

$$\theta_{\text{out}} = (\theta_{\text{in}} + 180^\circ) \bmod 360^\circ$$

注意： v4 の光ポートは専用の Python ポートオブジェクトではなく、レイアウト

上の Path と Text です。外部ツールで認識する際は、名前・座標・方向の抽出規則を統一する必要があります。

4.7 導波路長と光学位相

Straight の中心線長は、そのまま PCell の length です。

$$\boxed{L_{\mathrm{WG}} = L}$$

一様な単一モード導波路を仮定すると、伝搬位相は

$$\boxed{\phi = \beta L = \frac{2\pi n_{\mathrm{eff}}}{\lambda} L}$$

で表されます。ここで n_eff はモードの実効屈折率、λ は真空波長です。**v4 の PCell は n_eff を計算しません。** 位相や損失を求めるには、対象プロセスのモード解析・測定データが必要です。

4.8 KLayout での操作例

1. KLayout を起動し、v4 マクロが読み込まれていることを確認します。
2. PCell ライブラリ **Photonics** から **Straight** を選択します。
3. length = 100.0、width = 0.5 を指定します。
4. セル内に配置し、導波路レイヤー **1/0** と Pin レイヤー **2/0** を表示します。
5. 両端の opt1・opt2 の位置と向きを確認します。
6. length を変更し、PCell 形状と右端ポート位置が連動して変わることを確認します。

実際のメニュー名や配置手順は KLayout のバージョン・UI 設定により異なる場合があります。

4.9 設計・検証上の注意

▼表 4.3: パラメータ・仕様一覧

項目	確認内容
正の寸法	`length > 0`、`width > 0`
最小線幅	使用する製造プロセスの DRC に適合するか
DBU 精度	`width` が DBU に対して十分大きいか
接続幅	接続先ポートの導波路幅と一致するか
ポート方向	内向き・外向き角度を混同していないか
インスタンス変換	回転・反転・平行移動後のポート位置を正しく計算できるか
光学性能	損失・位相はレイアウト生成とは別途評価する

4.10 Photonics Router v6.x との連携

`Straight` は、Router のポート検出・接続判定の基本テストに適しています。例えば、2 つの Straight インスタンスを離して配置し、`Straight_A.opt2` と `Straight_B.opt1` を接続します。

▼text

```
Straight_A                        Straight_B
opt1 ●──────────────● opt2     opt1 ●──────────────● opt2
                  └──── Router ─────┘
```

Router 側では、各ポートの**インスタンス変換後の位置・方向・幅**を読み取る必要があります。ネットリストの概念的な表現例は次のとおりです（v4 の出力形式ではありません）。

▼text

```
Component: Straight_A
  type: Straight
  length_um: 100.0
  width_um: 0.5
  ports: [opt1, opt2]

Connection:
  from: Straight_A.opt2
  to:   Straight_B.opt1
```

4.11　v5 への改善候補

- `length`、`width` の妥当性検証と警告表示
- ポートの外向き方向を共通メタデータとして明示
- 中心線長・位相長のモデル連携
- テーパー等の幅変化デバイスとの接続規則の整備
- Router によるポート抽出・接続テストの自動化

これらは **v4 に実装済みの機能ではなく、今後の提案**です。

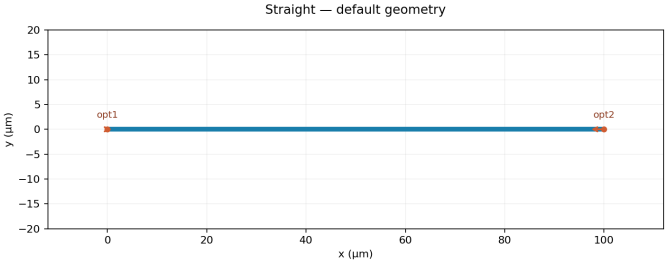
4.12　まとめ

`Straight` は最も基本的な 2 ポート PCell です。中心線は `(0,0)` から `(length,0)` までの直線で、初期値は**長さ 100 µ m、幅 0.5 µ m** です。両端の光ポートは Pin レイヤー上の Path と Text で表現されます。

Straight を基準として、座標変換、光ポート抽出、DBU 処理を検証することで、ほかの PCell や Photonics Router との接続処理を安定させられます。

次章： [第 5 章 Bend90 — 90°円弧導波路](05_bend90.md)（制作予定）

♣ PCell 構造図（v4 初期パラメータ）



▲図 4.1: 04_straight の中心線・MMI 領域・Optical Port

青：導波路中心線、淡青：*MMI* 領域、橙矢印：`add_pin()` の内向き方向。図は *Python* 実装の座標式に基づく概念図であり、*KLayout* の *GDS* レンダリングではありません。

第 5 章

Bend90 — 90°円弧導波路

実装基準：`pymacros/photonics_pdk_v4.lym`（https://github.com/TAKE-HooJoo/klayout-photonics-pdk/blob/main/pymacros/photonics_pdk_v4.lym）の `Bend90` PCell。**本章は既存の第 3 章に記録されたパラメータと 90°円弧の幾何学を基に作成した初稿です。現行 GitHub ソースの再取得はできていないため、以下の Python コードは動作原理を示す参考実装であり、v4 の逐語引用ではありません。ポート座標・角度・円弧の向きは公開前に必ず実装と照合してください。**

5.1 概要と用途

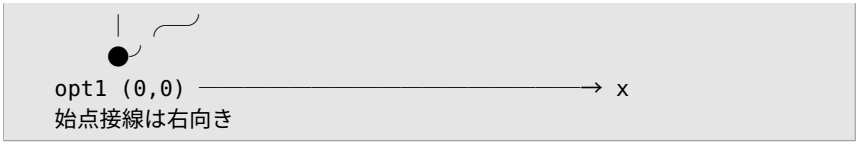
`Bend90` は、光導波路の進行方向を 90°変える曲げ導波路を生成する PCell です。直線導波路同士を直角方向に接続する際に利用します。PCell が生成するのは幾何学形状であり、曲げ損失や反射率を自動評価するものではありません。

5.2 基本構造と座標系

半径 `R` の四分円を用いる代表的な構成は次のとおりです。ここでは**説明用の座標系**として、始点を `(0,0)`、終点を `(R,R)`、円の中心を `(0,R)` とします。

▼ text

```
    y ↑
      |          ● opt2 (R,R)   ← 終点接線は上向き
      |       ╭──╯
      |    ╭──╯
```



この例の中心線は

$$x(\theta) = R\sin\theta, \qquad y(\theta) = R(1-\cos\theta), \qquad 0 \le \theta \le \frac{\pi}{2}$$

で表されます。始点は `(0,0)`、終点は `(R,R)`、始点接線は右向き、終点接線は上向きです。**実際の v4 が採用する回転方向・配置原点はソース確認が必要です。**

5.3　PCell パラメータ

既存の v4 アーキテクチャ資料に記録された値を以下に示します。

▼表 5.1: パラメータ・仕様一覧

パラメータ	型	初期値	単位	説明
`radius`	`TypeDouble`	10.0	μ m	円弧中心線の半径
`width`	`TypeDouble`	0.5	μ m	導波路幅
`points`	整数型	64	—	円弧を近似する点数
`layer`	`TypeLayer`	1/0	—	導波路レイヤー
`pin_layer`	`TypeLayer`	2/0	—	光ポートレイヤー

`points` が頂点数を意味するか分割数を意味するかは、v4 のループ境界で確認してください。`radius > 0`、`width > 0`、`points >= 2` は一般的な入力条件です。これらの検証が v4 に実装済みとは限りません。

5.4　Python による形状生成（参考実装）

以下は、**v4 からの引用ではなく**、上記の座標系で四分円を生成するための独立した参考コードです。`path_um()` と `add_pin()` は第 2・4 章の共通関数を想定しています。

▼ python

```
import math

def quarter_circle_points(radius, n=64):
    if radius <= 0 or n < 2:
        raise ValueError("radius > 0 and n >= 2 required")
    return [
        (radius * math.sin((math.pi / 2) * i / (n - 1)),
         radius * (1 - math.cos((math.pi / 2) * i / (n - 1))))
        for i in range(n)
    ]

# produce_impl() 内での使用例
pts = quarter_circle_points(self.radius, self.points)
cell.shapes(wg_layer).insert(path_um(pts, self.width, layout.db
›u))
```

`pya.Path` は点列を結ぶ折れ線形状を持ちます。円弧は有限個の線分で近似されるため、分割数と DBU の両方が形状精度に影響します。

5.5　円弧長の計算

円弧の長さは、半径と中心角（ラジアン）の積です。

$$L = R\theta$$

90°では　θ　= π /2 なので、

$$\boxed{L_{\text{bend}} = \frac{\pi R}{2}}$$

初期値 R=10 μ m では、

$$\boxed{L_{\text{bend}} \approx 15.708\ \mu\text{m}}$$

となります。これは**理想円弧の中心線長**です。実際の PCell の折れ線長はこれよりわずかに短く、DBU 丸めによる誤差も加わります。

5.6　分割数と円弧近似誤差

N 個の頂点を等角度で配置し、N − 1 本の線分で近似する場合、1 区間の角度は

$$\Delta\theta = \frac{\pi/2}{N-1}$$

です。各線分の長さは 2R sin(Δθ /2) なので、折れ線の合計長は

$$L_N = 2R(N-1)\sin\left(\frac{\pi}{4(N-1)}\right)$$

です。理想円弧との差は

$$\Delta L = \frac{\pi R}{2} - L_N \geq 0$$

となります。N=64、R=10 μ m の場合、DBU 丸め前の長さ誤差は約 **0.00041 μ m（0.41 nm）** です。実際の GDS での誤差は、さらに DBU に依存します。

最大弦偏差（円弧と線分の最大距離）は、

$$\boxed{e = R\left(1 - \cos\frac{\Delta\theta}{2}\right)}$$

です。分割数は、要求する図形精度とデータ量のバランスで決めます。

5.7　曲率と曲げ損失

理想円弧の曲率は一定で、

$$\boxed{\kappa = \frac{1}{R}}$$

です。R=10 μ m の場合、曲率は 0.1 μ m⊠[1] です。曲げ半径が小さいほど一般に放射損失の影響が大きくなりますが、具体的な損失は導波路断面、偏光、波長、クラッドなどに依存します。**v4 の Bend90 が損失値を保証するわけではありません。**

円弧と直線を直接接続すると、接線方向は一致しても曲率が 0 → 1/R に変化します。曲率の連続性が重要な場合は、EulerBend などの利用を検討します。

5.8　光ポートの定義

光ポートには中心座標、幅、名前、向きが必要です。上記の**参考座標系**での外向き方向は次のとおりです。

▼表 5.2: パラメータ・仕様一覧

ポート	参考位置	導波路に沿った伝搬方向	外向き方向
opt1	(0,0)	始点から右向き	180°
opt2	(R,R)	終点へ上向き	90°

add_pin() が受け取る inward_angle は外向き方向から 180°反転した値です。従って、この参考例なら opt1=0° 、opt2=270° となります。**この表は v4 のポート座標・角度を断定するものではありません。**

5.9　KLayout での確認手順

1. KLayout で Photonics ライブラリの Bend90 を配置します。
2. radius=10.0 μ m、width=0.5 μ m、points=64 を指定します。
3. 導波路レイヤー 1/0 と Pin レイヤー 2/0 を表示します。
4. 2 つのポートの名前・位置・向きを確認します。
5. radius を変更して、曲線形状とポート位置の変化を確認します。
6. points を増減させ、円弧の滑らかさと図形数を比較します。

5.10　Photonics Router v6.x との連携

Router が Bend90 インスタンスのポートを取得する際には、PCell ローカル座標をインスタンス変換（回転・反転・平行移動）でレイアウト座標に変換する必要があります。方向ベクトルにも回転・反転を適用し、ポート幅の整合性を検証します。

導波路長をネットリストへ格納する場合は、**理想円弧長 π R/2 と GDS 上の折れ線長を区別**してください。形状生成方式が変更された場合、後者が実際のレイアウトとの整合性を取りやすくなります。

5.11　v5 への改善案

▼表 5.3: パラメータ・仕様一覧

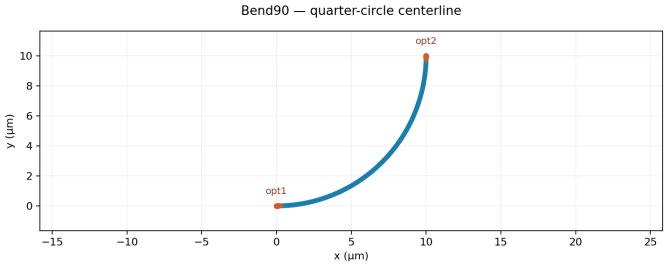
項目	改善案
パラメータ検証	`radius`、`width`、`points` の範囲チェック
形状精度	最大弦偏差を基準に分割数を自動設定
ポート仕様	共通の名前・方向・幅・座標表現に統一
導波路長	理想長と実レイアウト長を明示的に管理
曲率連続性	EulerBend との使い分けを明確化
光学評価	外部シミュレーションによる曲げ損失モデル連携

5.12　まとめ

Bend90 は、90°の進行方向変更を実現する基本　PCell です。理想円弧の長さは `πR/2`、曲率は `1/R` で求められます。KLayout 上では点列による折れ線近似と DBU 丸めが加わるため、精度管理が重要です。**正式公開前には v4 の実装に照らし、円弧の向き・端点座標・光ポート角度・点列生成方法を最終確認します。**

次章： [第 6 章 EulerBend — Euler 型曲げ導波路](06_euler_bend.md)（制作予定）

♣ PCell 構造図（v4 初期パラメータ）



▲図 5.1: 05_bend90 の中心線・MMI 領域・Optical Port

青：導波路中心線、淡青：*MMI* 領域、橙矢印：`add_pin()` の内向き方向。図は

Python 実装の座標式に基づく概念図であり、*KLayout* の *GDS* レンダリングではありません。

第 6 章

EulerBend — Euler 型曲げ導波路

実装基準：`pymacros/photonics_pdk_v4.lym`（https://github.com/TAKE-HooJoo/klayout-photonics-pdk/blob/main/pymacros/photonics_pdk_v4.lym）。本章は既存の第 3 章に記録された v4 仕様と曲線理論を基にした**技術解説書の初稿**です。GitHub の現行ソースをこの作成時点で再取得できなかったため、コード例は**参考実装**であり、v4 の逐語引用ではありません。特に曲線生成アルゴリズム、ポート角度、正規化処理は公開前にソース照合してください。

ソース照合追記（2026-10-15）：ユーザー提供の v4 ソースを静的解析した。実装は `n=max(32,int(points))` の中点法で、正規化パラメータ `u` に対して `k=4u`（前半）および `k=4(1-u)`（後半）を使用する。接線角を `th += (π/2)*k/n` で積算し、`cos(th), sin(th)` で座標を逐次更新した後、終点が `(radius,radius)` となるよう X/Y を別々にスケーリングする。したがって**厳密な Fresnel 積分によるクロソイドではなく、数値積分・異方的スケーリングによる Euler 型近似**である。光学損失や曲率連続性の保証はない。KLayout 実行検証は未実施。

6.1 概要と用途

`EulerBend` は、曲率が経路に沿って変化する Euler（Clothoid、クロソイド）型の曲げ導波路を生成する PCell です。直線と一定半径の円弧を直接接続すると、接続位置で曲率が 0 から `1/R` へ不連続に変化します。Euler 型曲線は曲率を徐々に増減させることで、この不連続を緩和する設計に使われます。

ただし、**Euler 型という名称だけで低損失が保証されるわけではありません**。光

学性能は導波路断面、偏光、波長、曲率分布、曲げ長、製造条件に依存し、電磁界シミュレーションによる確認が必要です。

6.2　Bend90 との比較

▼表 6.1: パラメータ・仕様一覧

比較項目	Bend90	EulerBend
代表的な中心線	四分円	曲率が変化する曲線
曲率	円弧上で一定 1/R	経路に沿って変化
直線との接続	一般に曲率が不連続	適切な設計で曲率を 0 から開始可能
中心線長	π R/2（理想円弧）	曲線形状から算出
設計上の注意	小半径で曲げ損失	曲率ピーク・占有面積・曲線正規化

6.3　v4 のパラメータ

既存の第 3 章に記録されている値を示します。

▼表 6.2: パラメータ・仕様一覧

パラメータ	型	初期値	単位	意味
radius	TypeDouble	10.0	μ m	v4 で曲線終点を合わせるフットプリント寸法
width	TypeDouble	0.5	μ m	導波路幅
points	整数型	96	—	曲線の離散化点数に関する指定
layer	TypeLayer	1/0	—	導波路レイヤー
pin_layer	TypeLayer	2/0	—	光ポートレイヤー

重要：v4 の radius は、最小曲率半径そのものではありません。 第 3 章で整理したとおり、生成曲線を X/Y 方向にスケーリングして終点を (radius, radius) に合わせる実装です。異方的なスケーリングは曲率分布を変えるため、入力値と実際の最小曲率半径を混同しないでください。

6.4　Euler 曲線の数学的原理

弧長を s、曲率をκ (s) とすると、理想的な Euler 曲線の基本的な特徴は

$$\boxed{\kappa(s) = as}$$

のように曲率が弧長に比例することです（a は曲率変化率）。接線角度θ (s) は

$$\frac{d\theta}{ds} = \kappa(s), \qquad \theta(s) = \theta_0 + \frac{a}{2}s^2$$

となります。始点を原点、初期接線を X 軸方向とすれば、中心線座標は

$$x(s) = \int_0^s \cos\!\left(\frac{a}{2}u^2\right) du, \qquad y(s) = \int_0^s \sin\!\left(\frac{a}{2}u^2\right) du$$

です。これは Fresnel 積分に関連します。

♣ Fresnel 積分

標準形は

$$C(t) = \int_0^t \cos\!\left(\frac{\pi}{2}u^2\right) du, \qquad S(t) = \int_0^t \sin\!\left(\frac{\pi}{2}u^2\right) du$$

です。変数変換によって Euler 曲線の座標を計算できます。

6.5　90°曲げを構成する方法

代表的な**対称 Euler Bend** では、前半で曲率を 0 から最大値まで増加させ、後半で 0 へ減少させます。

▼ text

```
直線 ―― 曲率増加 ―― 最大曲率 ―― 曲率減少 ―― 直線
          κ : 0 → κ max → 0
```

全長を L とする対称な例では、

$$\kappa(s) = \begin{cases} as & 0 \le s \le L/2 \\ a(L-s) & L/2 < s \le L \end{cases}$$

と定義できます。曲げ角は曲率の積分なので、

$$\Delta\theta = \int_0^L \kappa(s)ds = \frac{aL^2}{4}$$

です。90°曲げを要求すると、

$$\Delta\theta = \frac{\pi}{2} \quad \Rightarrow \quad \boxed{a = \frac{2\pi}{L^2}}$$

となります。この構成での最大曲率は

$$\kappa_{\max} = aL/2 = \pi/L, \qquad \boxed{R_{\min} = L/\pi}$$

です。**これは説明用の対称 Euler 曲線モデルに対する式であり、v4 の `radius` パラメータとの等式ではありません。**

6.6　v4 のフットプリントと曲率半径

既存の v4 仕様では、終点を

$$\boxed{P_{\text{end}} = (R, R)}$$

に合わせるための座標スケーリングが用いられています（`R = radius`）。そのため、`radius=10 μ m` のとき、終点位置は `(10 μ m, 10 μ m)` と整理できます。

一方、任意の平面曲線の曲率は

$$\boxed{\kappa(t) = \frac{|x'(t)y''(t) - y'(t)x''(t)|}{\left(x'(t)^2 + y'(t)^2\right)^{3/2}}}$$

で与えられます。曲率がゼロでない箇所の曲率半径は `R(t)=1/κ (t)` です。実際の `R_min` を知るには、**スケーリング後の中心線**に対してこの計算を行う必要があります。微分ベクトルがゼロになる特異点にも注意が必要です。

6.7　数値生成の参考コード

次のコードは **v4 の引用ではありません**。対称な曲率分布を弧長方向に積分し、ポリラインを生成する独立した参考実装です。返す座標はμ m 単位です。

▼python
```
import math

def symmetric_euler_90(length_um=30.0, points=97):
    """対称Euler 90°曲げの参考点列。 v4の実装そのものではない。"""
    if length_um <= 0 or points < 3:
        raise ValueError("length_um > 0, points >= 3 required")

    ds = length_um / (points - 1)
    a = 2.0 * math.pi / (length_um ** 2)

    def curvature(s):
        return a * min(s, length_um - s)

    theta = 0.0
    x, y = 0.0, 0.0
    result = [(x, y)]
    for i in range(1, points):
        s_mid = (i - 0.5) * ds
        theta_mid = theta + 0.5 * curvature(s_mid) * ds
        x += ds * math.cos(theta_mid)
        y += ds * math.sin(theta_mid)
        theta += curvature(s_mid) * ds
        result.append((x, y))
    return result

points_um = symmetric_euler_90(length_um=30.0)
# KLayoutへの挿入例（path_umは本ライブラリの共通関数を想定）:
# cell.shapes(wg_layer).insert(path_um(points_um, 0.5, layout.d›
›bu))
```

この例は数値積分の近似です。終点座標を `(radius, radius)` に合わせる処理は含みません。正確な終点、接線、曲率を同時に満たすには、設計変数の調整や数値最適化が必要です。

6.8　離散化と DBU

曲線を `points` 個の点列にすると、KLayout では隣接点間が線分で接続されます。分割数が少ない場合、形状誤差や導波路長誤差が増加します。さらに、`path_um()`

で DBU へ丸める際に、異なるサンプル点が同じ整数座標になることがあります。

検証では次を確認します。

1. 連続する点が重複していないか。
2. 曲線の始点・終点が意図したポート位置に一致するか。
3. 接線方向が期待する接続方向になっているか。
4. 近似線分の長さ・角度変化が許容範囲か。
5. DBU 丸め後に自己交差や不自然な折れが発生していないか。

6.9　導波路長の計算

理想的な曲線長は

$$L_{\mathrm{WG}} = \int \sqrt{\left(\frac{dx}{dt}\right)^2 + \left(\frac{dy}{dt}\right)^2}\, dt$$

です。点列の場合は

$$\boxed{L_{\mathrm{poly}} = \sum_{i=0}^{N-2} \sqrt{(x_{i+1} - x_i)^2 + (y_{i+1} - y_i)^2}}$$

で計算できます。`Bend90` の円弧長`π R/2` を、そのまま `EulerBend` の長さとして使うことはできません。

6.10　光ポートの考え方

EulerBend は 2 ポート導波路として扱います。v4 の共通 `add_pin()` では、Pin レイヤー上の短い Path と Text でポートを表現します。`inward_angle` は **PCell 内部へ向かう方向**であり、Router が外向き方向を用いる場合は

$$\theta_{\mathrm{out}} = (\theta_{\mathrm{in}} + 180^\circ) \bmod 360^\circ$$

で変換します。

v4 の**正確な `opt1`・`opt2` の角度と端点座標は、公開前に `produce_impl()` と照合してください**。図示や Router 連携では、内向き／外向きの混同を避けることが重要です。

6.11 KLayout での確認手順

1. `Photonics` ライブラリから `EulerBend` を選択します。
2. `radius=10.0`、`width=0.5`、`points=96` で配置します。
3. WG レイヤー（1/0）で中心線の形状を確認します。
4. Pin レイヤー（2/0）で `opt1`・`opt2` の位置と方向を確認します。
5. `radius` を変更し、フットプリントと曲線形状がどう変わるか比較します。
6. `points` を変更し、表示形状と DBU 丸めの影響を確認します。

※ 光損失の評価には KLayout 上の形状確認とは別に光学シミュレーションが必要です。

6.12 Photonics Router v6.x との連携

Router が EulerBend をインスタンスとして扱う場合、**配置変換後のポート座標と方向**を取得し、接続先と照合します。曲げ部分の導波路長をネットリストに記録する場合、v4 の `radius` から円弧長を推定せず、**実際の生成中心線から長さを算出**する必要があります。

推奨する検証項目は次のとおりです。

- インスタンス回転・反転後のポート位置と角度
- 接続部の位置誤差と導波路幅の一致
- 最小曲率半径と接線連続性
- 導波路長の計算精度
- 光ポートと Netlist 端子の対応

6.13 PCell Library v5 への改善案

6.14 まとめ

`EulerBend` は、曲率が徐々に変化する導波路形状を扱う PCell です。v4 の `radius` は終点フットプリントに関連する値であり、**最小曲率半径を直接保証しない**点が重要

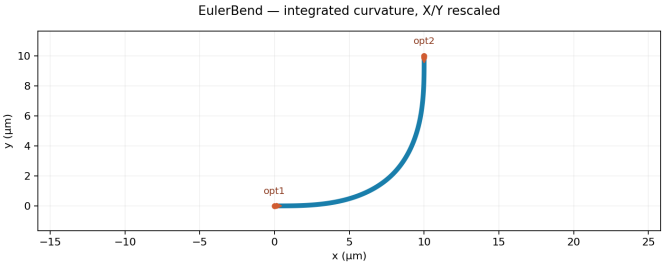
▼表 6.3: パラメータ・仕様一覧

改善項目	提案
パラメータ名	フットプリント寸法と最小曲率半径を区別する
曲率検証	生成後の `R_min` を算出して設計制約を確認
曲線生成	真の対称 Euler 曲線などの方式を明示
精度管理	点数だけでなく最大弦誤差を指定可能にする
ポート仕様	位置・幅・方向を共通メタデータ化
導波路長	中心線長を計算して Router へ提供
パラメータ検証	負値、極小寸法、丸めによる退化を検出

です。Bend90 との違いを理解し、曲率分布・導波路長・ポート方向を検証することで、Router 連携と次世代 PCell への拡張が容易になります。

次章：第 7 章 SBend — S 字導波路

♣ PCell 構造図（v4 初期パラメータ）



▲図 6.1: 06_euler_bend の中心線・MMI 領域・Optical Port

青：導波路中心線、淡青：*MMI* 領域、橙矢印：`add_pin()` の内向き方向。図は *Python* 実装の座標式に基づく概念図であり、*KLayout* の *GDS* レンダリングではありません。

第 7 章

SBend — S 字導波路

対象: `pymacros/photonics_pdk_v4.lym` の `class SBend(Base)` **確認日:** 2026-10-15 **位置づけ:** PCell Library v4 の実装仕様と、光学設計上の留意点を区別して解説する。

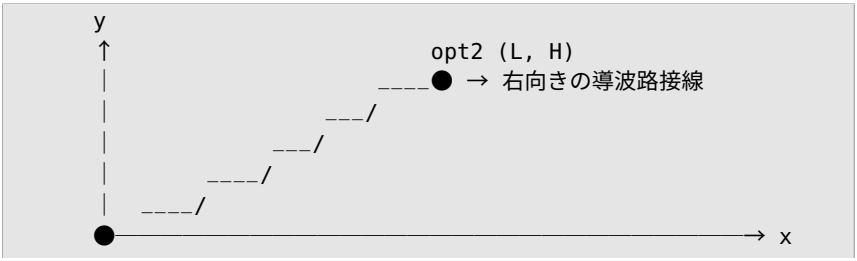
7.1 概要

`SBend` は、入口と出口の**接線方向を水平に保ったまま**、導波路の中心線を垂直方向へ `offset` だけ移動させる PCell である。隣接する光学部品のポート高さが異なる場合の接続や、導波路間隔の調整に使用できる。

v4 の SBend は **3 次 B é zier 曲線**で構成される。Euler 曲線や一定曲率の円弧とは異なる。

7.2 構造と座標

▼ text



```
opt1 (0,0)

L = length, H = offset
両端の導波路接線は水平
```

上図は概念図であり、曲線の正確な形状は後述の B é　zier 式で定義される。`offset < 0` なら下方向に移動する。

7.3　パラメータ

▼表 7.1: パラメータ・仕様一覧

パラメータ	型	v4 初期値	意味
`length`	`TypeDouble`	`20.0 μ m`	始点から終点までの X 方向距離 `L`
`offset`	`TypeDouble`	`5.0 μ m`	終点の Y 方向変位 `H`
`width`	`TypeDouble`	`0.5 μ m`	導波路幅
`points`	`TypeInt`	`64`	B é　zier 曲線の分割数（下限 16）
`layer`	`TypeLayer`	`(1, 0)`	導波路レイヤー（共通 Base）
`pin_layer`	`TypeLayer`	`(2, 0)`	光ポートレイヤー（共通 Base）

`length` は**中心線の実際の弧長ではない**。弧長は曲線形状に応じて長くなる。

7.4　3 次 B é　zier 曲線

制御点は、v4 のソースコードに従って次のように設定される。

$$P_0 = (0,0),\quad P_1 = (0.35L, 0),\quad P_2 = (0.65L, H),\quad P_3 = (L, H)$$

曲線は、パラメータ \(t\in[0,1]\) に対して

$$\boxed{B(t) = (1-t)^3 P_0 + 3(1-t)^2 t P_1 + 3(1-t) t^2 P_2 + t^3 P_3}$$

で表される。座標を展開すると

$$x(t) = L\{1.05(1-t)^2 t + 1.95(1-t)t^2 + t^3\},\qquad y(t) = H(3t^2 - 2t^3).$$

♣ 両端の接線

Bézier 曲線の導関数は

$$B'(0) = 3(P_1 - P_0) = (1.05L, 0), \quad B'(1) = 3(P_3 - P_2) = (1.05L, 0).$$

したがって **`length > 0` なら始点と終点の接線はともに $+\mathbf{X}$ 方向**である。これは水平導波路との滑らかな接続に有利である。一方、`length = 0` では端点の導関数がゼロになり、接線の定義が不安定になるため避ける。

7.5　v4 の Python 実装

以下は公開ソースの `SBend` に対応する主要処理である（読みやすさのため改行・コメントを整理）。

▼ python

```
class SBend(Base):
    def __init__(self):
        super(SBend, self).__init__()
        self.param("length", self.TypeDouble, "Length (um)", de›
›fault=20.0)
        self.param("offset", self.TypeDouble, "Vertical offset ›
›(um)", default=5.0)
        self.param("width", self.TypeDouble, "Width (um)", defa›
›ult=0.5)
        self.param("points", self.TypeInt, "Points", default=64›
›)
        self.layers()

    def produce_impl(self):
        l, c = self.layout, self.cell
        L, y = self.length, self.offset
        pts = bezier(
            (0, 0), (L * 0.35, 0),
            (L * 0.65, y), (L, y),
            max(16, int(self.points))
        )
        c.shapes(l.layer(self.layer)).insert(
            path_um(pts, self.width, l.dbu)
```

```
        )
        pin = l.layer(self.pin_layer)
        add_pin(c, l, pin, "opt1", 0, 0, 0, self.width)
        add_pin(c, l, pin, "opt2", L, y, 180, self.width)
```

共通関数 `bezier()` は `t=i/n`（`i=0,...,n`）の **`n+1` 個のサンプル点**を返す。`path_um()` は座標と幅を DBU へ変換し、`pya.Path` を生成する。

7.6 Optical Port

▼表 7.2: パラメータ・仕様一覧

ポート	座標（μ m）	add_pin に渡す角度	意味
opt1	(0, 0)	0°	左端から PCell 内部へ向かう方向は +X
opt2	(length, offset)	180°	右端から PCell 内部へ向かう方向は − X

注意: `add_pin()` の引数は `inward_angle`（PCell 内部を向く角度）である。出口の導波路接線が +X でも、`opt2` の**内向き**角度は 180°になる。外向き方向を扱う Router では角度を反転して解釈する必要がある。

7.7 導波路長

実際の中心線長は

$$S = \int_0^1 \sqrt{\left(\frac{dx}{dt}\right)^2 + \left(\frac{dy}{dt}\right)^2}\, dt$$

で求める。v4 の PCell はこの積分値をプロパティとして出力しない。数値積分、または離散化された中心線の点間距離の総和で評価する。

`length=20 μ m`, `offset=5 μ m` の初期設定では、S は 20 μ m より長い。正確な値は数値積分・量子化条件で変化する。

7.8　曲率と光学設計

平面曲線の曲率は

$$\kappa(t) = \frac{|x'(t)y''(t) - y'(t)x''(t)|}{\left(x'(t)^2 + y'(t)^2\right)^{3/2}}, \qquad R(t) = \frac{1}{\kappa(t)}.$$

端点の接線が水平であっても、**端点の曲率がゼロとは限らない**。この v4 実装では `offset ≠ 0` の場合、端点の曲率は一般に非ゼロであり、直線導波路との接続点で曲率が不連続となる。低損失設計を目指す場合は、最小曲率半径、曲率連続性、導波モードへの影響を別途評価する。

`length` を短くして `offset` を大きくすると、一般に曲げが急になる。レイアウト形状が生成できても、光学的な許容損失を満たすとは限らない。

7.9　離散化・DBU・入力値の注意

- `points` の実効値は `max(16, int(points))` である。
- 曲線は折れ線として `pya.Path` に渡される。サンプル数と DBU は形状精度に影響する。
- v4 には `length > 0`、`width > 0`、最小曲率半径などの入力検証がない。
- `offset = 0` の場合、幾何学的には水平な直線となる。
- `length < 0` では接線方向や光ポートの向きが v4 の固定角度と整合しなくなる可能性があるため、通常の設計では避ける。

7.10　KLayout での確認手順

1. `photonics_pdk_v4.lym` を KLayout の Python マクロとして読み込む。
2. PCell ライブラリー `Photonics` から `SBend` を選択する。
3. `length=20`, `offset=5`, `width=0.5`, `points=64` を設定する。
4. Waveguide レイヤー `(1,0)` と Pin レイヤー `(2,0)` を表示する。
5. 始点 `(0,0)`、終点 `(20,5)`、`opt1` と `opt2` の位置・向きを確認する。
6. `offset` を負に変更し、上下方向の反転を確認する。

7.11　Photonics Router v6.x との連携

SBend は、水平接線を持つ 2 ポートの導波路部品として Router の接続候補になる。ただし、v4 のポートは Pin レイヤー上の `Path` と `Text` で表現され、**構造化されたポートメタデータが PCell から直接返されるわけではない**。Router 側で名前、位置、内向き方向、幅、階層変換を正しく解釈する必要がある。

Router v6.x の個別バージョンでの自動認識・ネットリスト生成の挙動は、この章だけでは検証済みとはしない。

7.12　PCell Library v5 への改善案（未実装）

1. `length > 0`、`width > 0`、`points` の入力検証を追加する。
2. 最小曲率半径を算出し、設計ルール違反を検出する。
3. 中心線長を計算し、光路長評価に利用できるようにする。
4. 必要に応じて端点曲率ゼロの曲線方式を選択できるようにする。
5. 光ポートの位置・方向・幅を共通仕様で出力する。
6. 設計波長・導波路断面に基づく光学シミュレーションと照合する。

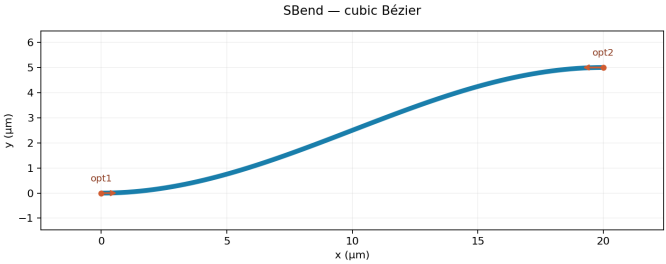
7.13　まとめ

`SBend` は、3 次 Bé zier 曲線を用いて水平接線を保ちながら導波路を上下に移動する PCell である。v4 の初期値は **`length=20 μ m`、`offset=5 μ m`、`width=0.5 μ m`、`points=64`**。光ポートは `opt1=(0,0)`、`opt2=(length,offset)` に置かれる。接線連続性と曲率連続性は異なるため、光学設計では最小曲率半径と接続損失の確認が重要となる。

参照実装: `photonics_pdk_v4.lym`（https://github.com/TAKE-HooJoo/klayout-photonics-pdk/blob/main/pymacros/photonics_pdk_v4.lym）の `bezier()`、`add_pin()`、`SBend`。

♣ PCell 構造図（v4 初期パラメータ）

青：導波路中心線、淡青：*MMI* 領域、橙矢印：***`add_pin()`*** の内向き方向。図は *Python* 実装の座標式に基づく概念図であり、*KLayout* の *GDS* レンダリングではありません。



▲図 7.1: 07_sbend の中心線・MMI 領域・Optical Port

第 8 章

Ring — リング共振器

対象実装: `pymacros/photonics_pdk_v4.lym` の `class Ring(Base)` **確認日:** 2026-10-15 **対象:** KLayout Silicon Photonics PDK v4

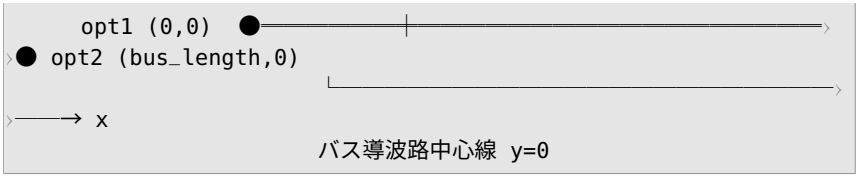
8.1 概要

`Ring` は、直線状のバス導波路と円環状の導波路を隣接配置する、**単一バス型（all-pass 型）リング共振器**のレイアウト PCell である。バス導波路の入力から光を入射し、リングの共振条件に応じた透過特性を得ることを想定している。v4 は形状生成用であり、透過スペクトル、結合係数、損失などを自動計算しない。

8.2 構造と座標系

▼ text

```
                    y ↑
                      |        リング中心 (cx, cy)
                      |            ◯
                      |         .-''''''-.
                      |        /          \
                      |        \          /
                      |         '-.____.-'
                      |             ↑
                      |         edge gap = gap
                      |
```



リングの中心と半径は、ソースコード上で次のように定義される。

$$c_x = \frac{\text{bus_length}}{2}, \qquad c_y = \text{radius} + \text{gap} + \text{width}.$$

したがってリング中心線の最下点は \(y=\mathrm{gap}+\mathrm{width}\) に位置する。バス導波路とリング導波路の中心線間隔は \(\mathrm{gap}+\mathrm{width}\)、両導波路の**外縁間の最短距離**は、両者が同じ幅である場合、指定した `gap` になる。

8.3　PCell パラメータ

▼表 8.1: パラメータ・仕様一覧

パラメータ	型	v4 初期値	内容
`radius`	`TypeDouble`	10.0 μ m	リング中心線の半径
`width`	`TypeDouble`	0.5 μ m	バス・リング共通の導波路幅
`gap`	`TypeDouble`	0.2 μ m	バスとリングの導波路外縁間のギャップ
`bus_length`	`TypeDouble`	40.0 μ m	バス導波路の長さ
`points`	`TypeInt`	128	リングを近似する線分数（実装下限 48）
`layer`	`TypeLayer`	(1, 0)	導波路レイヤー
`pin_layer`	`TypeLayer`	(2, 0)	Optical Port レイヤー

上記の初期値は公開 v4 の `Ring.__init__()` に基づく。物理的に成立する値域の検証は、実装に明示されていない。

8.4　Python 実装の構造

公開 v4 の処理を要約すると、次のとおりである。

▼python

```
# 1. バス導波路を生成
c.shapes(wg).insert(
    path_um([(0, 0), (self.bus_length, 0)], self.width, l.dbu)
)

# 2. リング中心を決定
cx = self.bus_length / 2
cy = self.radius + self.gap + self.width
n = max(48, int(self.points))

# 3. 円周を離散点列に変換
pts = [
    (
        cx + self.radius * math.cos(2 * math.pi * i / n),
        cy + self.radius * math.sin(2 * math.pi * i / n)
    )
    for i in range(n + 1)
]
c.shapes(wg).insert(path_um(pts, self.width, l.dbu))

# 4. バス導波路両端に光ポートを配置
add_pin(c, l, pin, "opt1", 0, 0, 0, self.width)
add_pin(c, l, pin, "opt2", self.bus_length, 0, 180, self.width)
```

上記は `produce_impl()` の主要処理を抜粋・整形したものである。リングは `i=0` から `n` までの `n+1` 点で定義され、最後の点が最初の点と一致する閉曲線を `pya.Path` として描画する。**ポリゴンによる円環の差分生成ではない**点に注意する。

8.5　リング長と共振条件

理想的な円形中心線の一周長は

$$\boxed{L_{\mathrm{ring}} = 2\pi R}$$

である。初期値 \(R=10\,\mu\mathrm{m}\) では \(L_\mathrm{ring}\approx62.832\,\mu\mathrm{m}\) となる。

リング内の伝搬位相が整数倍の \(2\pi\) になると共振が起こる。

$$\beta(\lambda_m) L_{\mathrm{ring}} = 2\pi m, \qquad \beta = \frac{2\pi n_{\mathrm{eff}}(\lambda)}{\lambda}.$$

したがって

$$\boxed{m\lambda_m = n_{\mathrm{eff}}(\lambda_m) L_{\mathrm{ring}}}$$

である。ただし、実際の共振波長を求めるには、導波路断面と材料に依存する有効屈折率 \(n_\mathrm{eff}\) が必要である。**半径だけでは共振波長を確定できない**。

8.6　FSR（自由スペクトル間隔）

隣接する共振波長間隔は、波長領域で近似的に

$$\boxed{\mathrm{FSR}_\lambda \approx \frac{\lambda^2}{n_g L_{\mathrm{ring}}}}$$

と表される。\(n_g\) は群屈折率である。たとえば、**説明用の仮定**として \(\lambda=1.55\,\mu\mathrm{m}\)、\(n_g=4.0\)、\(R=10\,\mu\mathrm{m}\) とすると、

$$\mathrm{FSR}_\lambda \approx \frac{1.55^2}{4.0 \times 2\pi \times 10}\,\mu\mathrm{m} \approx 0.00956\,\mu\mathrm{m} = 9.56\,\mathrm{nm}.$$

この値は**シミュレーションや測定による検証値ではない**。

8.7　ギャップと結合

gap はバス導波路とリング導波路のエバネッセント結合に影響する重要なパラメータである。一般に、他の条件が等しければギャップが小さいほど結合が強くなる傾向があるが、結合係数は幅、層厚、屈折率、波長、偏波、曲率などにも依存する。

v4 では gap を変更すると**レイアウト上の位置関係のみ**が更新され、結合係数や臨界結合条件は計算されない。結合設計にはモード解析や電磁界シミュレーションが必要である。

8.8　円周の離散化と誤差

`points = N` としたとき、リング中心線は \(N\) 本の弦で近似される。理想円と各弦の最大偏差（サジッタ）は

$$\boxed{\delta = R\left(1 - \cos\frac{\pi}{N}\right)}$$

である。初期値 \(R=10\,\mu\mathrm{m}\)、\(N=128\) では \(\delta\approx0.00301\,\mu\mathrm{m}=3.01\,\mathrm{nm}\)。実際の座標はさらに DBU で丸められるため、最終的な形状誤差は DBU にも依存する。

離散化された中心線長は

$$L_N = 2NR\sin\left(\frac{\pi}{N}\right)$$

であり、理想円周長 \(2\pi R\) よりわずかに短い。

8.9　Optical Port の定義

▼表 8.2: パラメータ・仕様一覧

ポート	座標 (μ m)	add_pin の内向き角度	接続対象
opt1	(0, 0)	0°	バス左端
opt2	(bus_length, 0)	180°	バス右端

リング自体には独立した光ポートが定義されていない。v4 の `add_pin` は Pin レイヤーに短い Path と名前の Text を配置する。**角度は素子内部を向く方向**として扱う。

8.10　KLayout での確認手順

1. PDK v4 マクロを KLayout に読み込み、PCell ライブラリ `Photonics` の `Ring` を配置する。
2. `radius=10`, `width=0.5`, `gap=0.2`, `bus_length=40`, `points=128` を確認する。
3. Waveguide レイヤー `(1,0)` と Pin レイヤー `(2,0)` を表示する。

4. バス両端の `opt1` / `opt2` を確認する。
5. `gap` を変更してリング位置が変化すること、`points` を変更して多角形近似が変わることを確認する。
6. 意図しない重なりや製造ルール違反がないか、別途 DRC で確認する。

8.11　Photonics Router v6.x との連携

Ring の外部接続対象はバス導波路の 2 ポートである。Router 側で接続判定を行う場合、インスタンスの配置変換を考慮し、ポート位置と向きをグローバル座標へ変換する必要がある。

重要: v4 の Pin レイヤー上の Path/Text は、Router が自動的に認識することを保証しない。ポート抽出方式と命名規則が一致するか、Router 側で検証する必要がある。また、リングとバスの間は幾何学的に非接触でも光学的に結合するため、単純な接触判定によるネットリストでは結合を表現できない。

8.12　v5 への改善提案

- `radius > 0`、`width > 0`、`gap >= 0` などの入力検証を追加する。
- リング導波路幅とバス導波路幅を独立パラメータにする。
- 円周近似の最大許容誤差から `points` を自動決定できるようにする。
- リング共振器の結合関係をネットリスト上で表現する仕組みを追加する。
- `n_eff`、`n_g`、損失、結合係数を外部モデルと関連付ける。ただし、レイアウト PCell と光学モデルの責務を明確に分離する。
- add-drop 型を別 PCell として追加する。

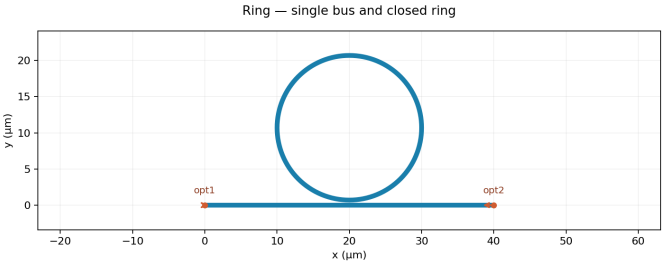
これらは**今後の改善案**であり、v4 に実装されている機能ではない。

8.13　まとめ

`Ring` はバス導波路と円形リングを生成する単一バス型共振器 PCell である。主要な設計値はリング中心線半径 `radius`、導波路幅 `width`、外縁間隔 `gap`、バス長 `bus_length` である。v4 は形状と 2 つの光ポートを生成するが、共振波長・FSR・結合係数を確定するには別途光学モデルが必要となる。

参照実装: photonics_pdk_v4.lym (https://github.com/TAKE-HooJoo/klayout-photonics-pdk/blob/main/pymacros/photonics_pdk_v4.lym)（`Ring` クラス）

♣ PCell 構造図（v4 初期パラメータ）



▲図 8.1: 08_ring の中心線・MMI 領域・Optical Port

青：導波路中心線、淡青：*MMI* 領域、橙矢印：`add_pin()` の内向き方向。図は *Python* 実装の座標式に基づく概念図であり、*KLayout* の *GDS* レンダリングではありません。

第 9 章

DirectionalCoupler — 方向性結合器

対象実装: pymacros/photonics_pdk_v4.lym の class DirectionalCoupler(Base) **確認日:** 2026-10-15 **対象:** KLayout Silicon Photonics PDK v4

9.1 概要

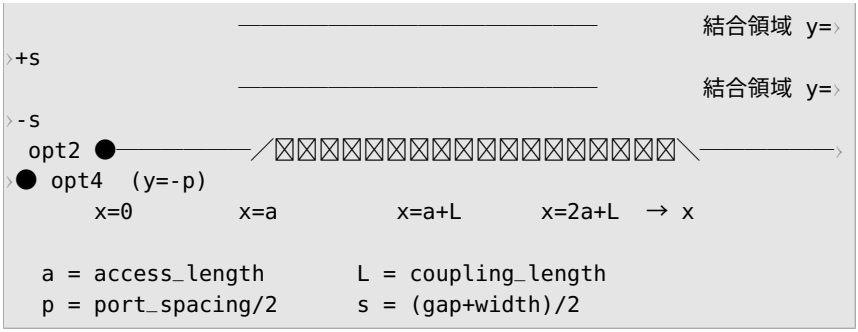
DirectionalCoupler は、近接して並走する 2 本の光導波路間のエバネッセント結合を利用する、**4 ポートの方向性結合器**のレイアウト PCell である。中央の直線結合領域と、その両側の S 字アクセス導波路から構成される。分岐器、光スイッチ、干渉計などの構成要素として利用できる。

重要: v4 が生成するのは幾何形状と光ポートであり、所望の 50:50 分岐比や結合効率を保証するものではない。結合係数は導波路断面、材料、波長、偏波、ギャップ等に依存する。

9.2 構造と座標系

▼ text

```
                  y ↑
  opt1 ●──────────╲__________________╱──────────● opt3  (y=+
p)
```



上図は**概念図**であり、実際の曲線は 3 次 Bézier 曲線である。上下の導波路は x 軸に関して対称で、左端の 2 ポートから中央の結合領域へ近づき、右端で再び離れる。

中央部の導波路中心線間隔は

$$d_{\text{center}} = 2s = \text{gap} + \text{width}.$$

両導波路の幅が `width` であるため、中央直線部の**エッジ間ギャップ**は `gap` となる（`gap` ≥ `0` かつ正の幅を想定）。

9.3　PCell パラメータ

▼表 9.1: パラメータ・仕様一覧

パラメータ	型	v4 初期値	内容
`coupling_length`	`TypeDouble`	20.0 μm	中央直線結合部の長さ
`access_length`	`TypeDouble`	15.0 μm	片側 S 字アクセス部の x 方向長さ
`gap`	`TypeDouble`	0.2 μm	中央直線部の導波路エッジ間隔
`port_spacing`	`TypeDouble`	5.0 μm	左右端の上下ポート中心間隔
`width`	`TypeDouble`	0.5 μm	両導波路共通の幅
`layer`	`TypeLayer`	(1, 0)	導波路レイヤー（共通 `Base` 由来）
`pin_layer`	`TypeLayer`	(2, 0)	光ポートレイヤー（共通 `Base` 由来）

v4 ではアクセス部の Bézier 曲線は**各 48 分割に固定**されており、曲線分割数を PCell パラメータから変更する仕組みはない。

初期値の場合、`a=15`, `L=20`, `p=2.5`, `s=0.35`（すべてμm）で、全体の x 方向長さは

$$X = 2a + L = 50\ \mu\ \mathrm{m}.$$

9.4　Python ソースコードの解説

以下は公開 v4 の `produce_impl()` にある主要処理を、読みやすく改行したものである。

▼python

```
l, c = self.layout, self.cell
wg = l.layer(self.layer)
pin = l.layer(self.pin_layer)

a = self.access_length
L = self.coupling_length
s = (self.gap + self.width) / 2
p = self.port_spacing / 2

top = (
    bezier((0, p), (a * .35, p), (a * .65, s), (a, s), 48)
    + [(a + L, s)]
    + bezier(
        (a + L, s),
        (a + L + a * .35, s),
        (a + L + a * .65, p),
        (2 * a + L, p),
        48
    )[1:]
)
bot = [(x, -y) for x, y in top]

c.shapes(wg).insert(path_um(top, self.width, l.dbu))
c.shapes(wg).insert(path_um(bot, self.width, l.dbu))

x = 2 * a + L
add_pin(c, l, pin, "opt1", 0, p, 0, self.width)
add_pin(c, l, pin, "opt2", 0, -p, 0, self.width)
add_pin(c, l, pin, "opt3", x, p, 180, self.width)
add_pin(c, l, pin, "opt4", x, -p, 180, self.width)
```

処理は次の 4 段階に整理できる。

1. `s` と `p` を計算し、結合部と外部ポートの上下位置を決める。
2. 上側導波路 `top` を、入力側 Bézier 曲線・中央直線・出力側 Bézier 曲線の点列として構築する。
3. `bot=[(x,-y) for x,y in top]` によって下側導波路を x 軸対称に生成する。
4. 2 本の導波路を描画し、4 つの名前付き光ポートを生成する。

`bezier(...)[1:]` は、出力側曲線の最初の点が中央直線部の終点と重複するのを避ける処理である。入力側 Bézier 曲線は 49 点、中央直線の終点が 1 点、出力側曲線の追加分が 48 点なので、上側中心線の点列は**合計 98 点**となる。

9.5　3 次 Bézier アクセス導波路

入力側の上側導波路は、次の 4 制御点を用いる。

$$P_0 = (0, p), \quad P_1 = (0.35a, p), \quad P_2 = (0.65a, s), \quad P_3 = (a, s).$$

3 次 Bézier 曲線は

$$B(t) = (1-t)^3 P_0 + 3(1-t)^2 t P_1 + 3(1-t) t^2 P_2 + t^3 P_3, \qquad 0 \le t \le 1.$$

端点での接線は

$$B'(0) = 3(P_1 - P_0) = (1.05a, 0),$$

$$B'(1) = 3(P_3 - P_2) = (1.05a, 0).$$

したがって `a>0` なら、アクセス曲線の両端は水平方向を向く。中央直線部と接続する箇所でも**接線方向が連続**する。一方、**曲率まで連続するとは限らない**。低損失設計の観点では、接続点の曲率変化と最小曲率半径の検証が別途必要である。

9.6　Optical Port の定義

初期値では以下となる。

▼表 9.2: パラメータ・仕様一覧

ポート	座標 (μ m)	inward_angle	接続方向の解釈
opt1	(0, +p)	0°	左上、セル内向きは +x
opt2	(0, -p)	0°	左下、セル内向きは +x
opt3	(2a+L, +p)	180°	右上、セル内向きは − x
opt4	(2a+L, -p)	180°	右下、セル内向きは − x

▼text

```
opt1 = ( 0, +2.5) μm  inward=0°
opt2 = ( 0, -2.5) μm  inward=0°
opt3 = (50, +2.5) μm  inward=180°
opt4 = (50, -2.5) μm  inward=180°
```

共通関数 `add_pin()` は、Pin レイヤーに短い `pya.Path` と `pya.Text` を配置する。角度は**外向き法線ではなく、ポートからセル内部へ向かう方向**である点に注意する。v4 のポートはテキストと形状であり、明示的なモード番号・偏波・光学ネットリスト情報は保持していない。

9.7　光結合の理論

同一の単一モード導波路 2 本からなる理想的な、損失のない対称方向性結合器を仮定する。中央の平行結合区間において、結合係数を κ [1/μ m]、長さを L とすると、単一ポートから入射した光の概略的なパワー比は

$$P_{\text{through}} = \cos^2(\kappa L), \qquad P_{\text{cross}} = \sin^2(\kappa L).$$

理想的な 50:50 分岐の条件の一つは

$$\kappa L = \frac{\pi}{4} \pmod{\pi/2}$$

である。**ただし、これはアクセス領域での結合を無視した簡略モデル**であり、実際にはアクセス部でも導波路が近づくため結合が発生し得る。結合領域全体を含むモード解析・EME・FDTD 等による検証が必要となる。

導波路間の伝搬定数不整合 Δβ　がある場合には、完全結合を前提とした上式をそのまま適用できない。結合係数 κ　と Δβ　を用いる結合モード理論や、偶・奇スーパーモードの実効屈折率から結合長を評価する。

$$L_\pi = \frac{\pi}{|\beta_{\mathrm{even}} - \beta_{\mathrm{odd}}|}$$

は、理想的な同一導波路のスーパーモード解析における、入射導波路から反対側へパワーが最大移行する長さの一例である（定義の取り方によって「結合長」の呼称が異なるため注意）。

9.8　導波路長と光学位相

各経路の中心線長は、入力アクセス部・中央直線部・出力アクセス部の合計である。

$$L_{\mathrm{path}} = L + 2L_{\mathrm{access}}, \qquad L_{\mathrm{access}} = \int_0^1 \|B'(t)\|\, dt.$$

`access_length` はアクセス部の **x 方向長さ**であり、実際の曲線長ではない。上下経路は鏡映対称なので、理想的な幾何学的中心線長は等しい。ただし、曲線の離散化と DBU 丸めによって微小な誤差が生じ得る。

9.9　離散化と DBU

`bezier()` は `n=48` に対して `n+1=49` 点を返す。`path_um()` は各座標を `q(v,dbu)=int(round(v/dbu))` で整数 DBU 座標に変換し、`pya.Path` を生成する。したがって、細かな形状の再現性は DBU とサンプリング点数に依存する。

設計値に対してギャップが小さい場合、**DBU 丸めによる実効ギャップの変化**を確認すること。曲線が十分に細かく近似されているかは、描画表示だけでなく生成された形状の測定によって評価する。

9.10　KLayout での確認手順

1. `photonics_pdk_v4.lym` を KLayout のマクロとして読み込み、`Photonics` ライブラリが登録されていることを確認する。
2. `DirectionalCoupler` PCell を配置し、初期値で 2 本の導波路が左右対称に接続されているか確認する。

3. レイヤー `(1,0)` に導波路、`(2,0)` に `opt1`〜`opt4` の Pin とラベルが生成されているか確認する。
4. `gap` を変更し、中央結合部のエッジ間隔が期待どおりに変わるか確認する。
5. `port_spacing` と `access_length` を変更し、アクセス曲線の形状・曲率・重なりの有無を確認する。
6. 必要に応じて DRC、導波路断面解析、結合効率の電磁界シミュレーションを実施する。

9.11 Photonics Router v6.x との連携

v4 の `DirectionalCoupler` は、4 つのポート位置と向きを Pin レイヤー上に明示するため、**ポート検出型 Router に接続するための基本情報**を備える。ただし、Router が実際に認識できるかどうかは、Router 側のレイヤー設定・Text 解釈・階層変換・方向定義に依存する。

検証時には、次の事項を確認する。

- 4 ポートすべての名前・位置・角度が正しく抽出されること。
- インスタンスの回転・鏡映後にも、ポート座標と方向が正しく変換されること。
- `opt1`〜`opt4` の接続関係を、単なる交差や幾何学的接触と混同しないこと。
- 方向性結合器をネットリスト上で **4 端子の光学素子**として表現できること。

9.12 v4 の制約と v5 への改善案

▼表 9.3: パラメータ・仕様一覧

項目	v4 の現状	改善案（未実装）
分岐比	形状のみ、分岐比の保証なし	波長・偏波別の結合係数モデルを連携
曲線分割数	各アクセス部 48 固定	サンプリング数のパラメータ化
入力値検証	明示的な値域検証なし	正の幅・長さ、非負ギャップなどを検証
曲率	B é zier 形状、曲率制約なし	最小曲率半径チェックと最適化
光ポート	Pin Path + Text	ポートメタデータの明示的管理
ネットリスト	PCell 自体はモデルを持たない	4 ポート素子モデルとの対応付け
レイアウト検証	自動光学検証なし	DRC・EME/FDTD 結果との連携

特に `port_spacing` が中央の中心線間隔 `gap+width` より小さい場合、アクセス部が「中央へ近づく」形状とは限らない。パラメータの組合せに応じて、曲線形状と最小ギャップを検査することが望ましい。

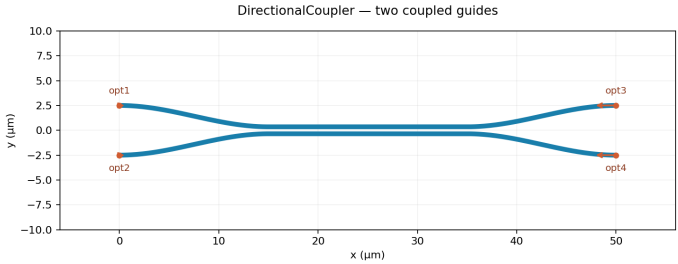
9.13　まとめ

`DirectionalCoupler` は、2 本の鏡映対称な導波路、中央直線結合部、左右の 3 次 Bé zier アクセス部、および 4 つの光ポートから構成される。初期値では全長 50 µ m、中央エッジ間ギャップ 0.2 µ m、外部ポート間隔 5 µ m となる。

設計上の核心は、幾何学的な `coupling_length` と `gap` を設定するだけでは、目標分岐比は確定しないという点である。実際の結合効率は導波路断面、モード、波長、アクセス部の影響を含めて評価する必要がある。

実装参照: `pymacros/photonics_pdk_v4.lym` (https://github.com/TAKE-HooJoo/klayout-photonics-pdk/blob/main/pymacros/photonics_pdk_v4.lym) の `DirectionalCoupler`、共通関数 `bezier()`、`path_um()`、`add_pin()`。

♣ PCell 構造図（v4 初期パラメータ）



▲図 9.1: 09_directional_coupler の中心線・MMI 領域・Optical Port

青：導波路中心線、淡青：*MMI* 領域、橙矢印：*`add_pin()`* の内向き方向。図は *Python* 実装の座標式に基づく概念図であり、*KLayout* の *GDS* レンダリングではありません。

第 10 章

MMI_1x2 — 多モード干渉型 1 × 2 光分岐器

対象実装: `pymacros/photonics_pdk_v4.lym` の `class MMI1x2(Base)`（ライブラリ登録名 `MMI_1x2`）**確認日:** 2026-10-15 **対象:** KLayout Silicon Photonics PDK v4

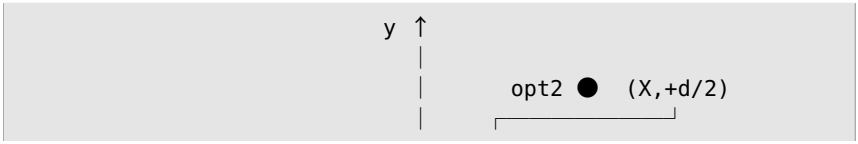
10.1 概要と用途

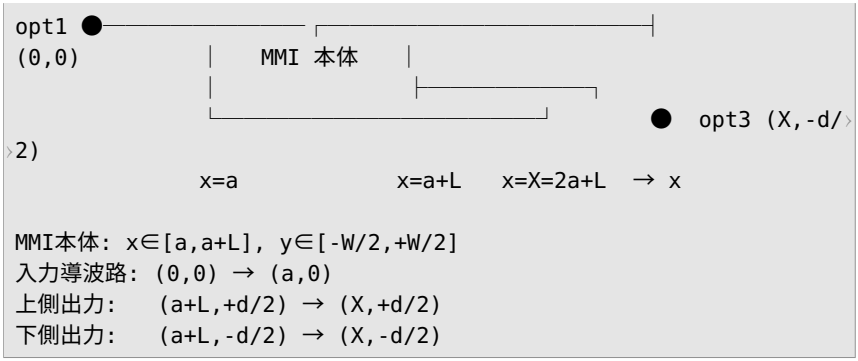
`MMI_1x2` は、幅の広い多モード導波路領域（MMI 本体）と、左側の入力導波路 1 本、右側の出力導波路 2 本からなる **1 入力 2 出力のレイアウト PCell** である。光分岐器、MZI の入力分岐部、干渉回路などの構成要素として利用できる。

重要: v4 は幾何形状と光ポートを生成する。50:50 分岐比、挿入損失、帯域幅などの光学性能は計算・保証しない。MMI 寸法はモード解析、EME/FDTD などで別途設計・検証する必要がある。

10.2 構造と座標系

▼ text





この図は接続関係を示す概念図であり、実際の MMI 本体は単一の矩形 `pya.Box`、アクセス導波路は 3 本の `pya.Path` で構成される。`a=access_length`、`L=mmi_length`、`W=mmi_width`、`d=port_spacing` とする。

10.3　PCell パラメータ

▼表 10.1: パラメータ・仕様一覧

パラメータ	型	v4 初期値	意味
`wg_width`	`TypeDouble`	0.5 μ m	入出力アクセス導波路の幅
`mmi_width`	`TypeDouble`	4.0 μ m	MMI 本体の幅（y 方向）
`mmi_length`	`TypeDouble`	20.0 μ m	MMI 本体の長さ（x 方向）
`port_spacing`	`TypeDouble`	2.0 μ m	出力 2 ポートの中心間隔
`access_length`	`TypeDouble`	5.0 μ m	入力・出力アクセス導波路の長さ
`layer`	`TypeLayer`	(1, 0)	導波路レイヤー（共通 Base）
`pin_layer`	`TypeLayer`	(2, 0)	光ポートレイヤー（共通 Base）

初期値では全長は `X = 2 × 5+20 = 30 μ m`、MMI 本体は `x=5〜25 μ m`、`y=-2〜+2 μ m`、出力中心は `y=±1 μ m` となる。

10.4　Python 実装の解説

以下は v4 ソースの処理を、変数名と構造を維持して読みやすく整形した抜粋である。

▼python

```
class MMI1x2(Base):
    def __init__(self):
        super(MMI1x2, self).__init__()
        self.param("wg_width", self.TypeDouble, "WG width (um)"
, default=0.5)
        self.param("mmi_width", self.TypeDouble, "MMI width (um
)", default=4.0)
        self.param("mmi_length", self.TypeDouble, "MMI length (
um)", default=20.0)
        self.param("port_spacing", self.TypeDouble, "Output spa
cing (um)", default=2.0)
        self.param("access_length", self.TypeDouble, "Access le
ngth (um)", default=5.0)
        self.layers()

    def produce_impl(self):
        l, c = self.layout, self.cell
        wg = l.layer(self.layer)
        pin = l.layer(self.pin_layer)
        a = self.access_length
        L = self.mmi_length
        y = self.port_spacing / 2

        c.shapes(wg).insert(
            pya.Box(q(a, l.dbu), q(-self.mmi_width / 2, l.dbu),
                    q(a + L, l.dbu), q(self.mmi_width / 2, l.db
u))
        )
        c.shapes(wg).insert(path_um([(0, 0), (a, 0)], self.wg_w
idth, l.dbu))
        c.shapes(wg).insert(path_um([(a + L, y), (2*a + L, y)],
 self.wg_width, l.dbu))
        c.shapes(wg).insert(path_um([(a + L, -y), (2*a + L, -y)
], self.wg_width, l.dbu))

        add_pin(c, l, pin, "opt1", 0, 0, 0, self.wg_width)
        add_pin(c, l, pin, "opt2", 2*a + L, y, 180, self.wg_wid
th)
        add_pin(c, l, pin, "opt3", 2*a + L, -y, 180, self.wg_wi
dth)
```

- `q(value, dbu)` は μ m 値を `round(value/dbu)` により整数 DBU へ変換する。
- `pya.Box` は MMI の矩形領域を生成する。
- `path_um` はアクセス導波路の中心線と幅から `pya.Path` を生成する。
- `add_pin` は Pin レイヤー上に短い Path とポート名 Text を作成する。
- Python クラス名は `MMI1x2`、KLayout の PCell 登録名は `MMI_1x2` であり、表記が異なる。

10.5 Optical Port の仕様

▼表 10.2: パラメータ・仕様一覧

ポート	位置 (μ m)	初期値の位置 (μ m)	内向き角度	用途
opt1	(0, 0)	(0, 0)	0°	入力側
opt2	(2a+L, +d/2)	(30, +1)	180°	上側出力
opt3	(2a+L, -d/2)	(30, -1)	180°	下側出力

`add_pin` の角度は**素子内部へ向かう方向**である。ポートの向きは接続判定に利用できるが、v4 の Pin は `pya.Text` と短い `pya.Path` であり、独立した構造化ポートオブジェクトではない。Router 側ではレイヤー、ラベル、方向を正しく読み取る必要がある。

10.6 MMI の光学原理（設計背景）

MMI 領域では複数の横モードが伝搬し、それぞれの伝搬定数の差により干渉パターンが変化する。適切な長さでは入力場の複数像が形成され、2 本の出力導波路に光を取り出せる（**自己結像**）。

モード間の位相差は概念的に

$$\Delta\phi_{mn}(L) = (\beta_m - \beta_n)L$$

で表される。ここで β _m と β _n は各モードの伝搬定数である。必要な MMI 長は有効幅、波長、偏波、材料、導波路断面、励起モードなどに依存するため、**`mmi_length=20 μ m` が一般に 50:50 分岐を意味するわけではない**。

MMI の分岐比はアクセス導波路の出力位置やモード重なりにも依存する。幾何学的に対称な出力配置は左右対称性を与えるが、低損失・高均等性の十分条件ではない。

10.7　寸法条件と幾何学的な注意点

1. `wg_width > 0`、`mmi_width > 0`、`mmi_length > 0`、`access_length > 0` が基本的な前提となる。
2. 出力アクセス導波路を MMI 矩形の上下端からはみ出させないための単純な条件は `port_spacing/2 + wg_width/2 ≤ mmi_width/2`、すなわち `port_spacing + wg_width ≤ mmi_width` である。ただしこれは光学的最適化条件ではない。
3. 出力 2 本を幾何学的に離すには `port_spacing ≥ wg_width` が目安となる。ギャップを設ける場合はより大きな間隔が必要。
4. v4 の実装には、これらの値を自動検証する `coerce_parameters_impl()` 等の処理は見当たらない。無効な寸法設定は利用者側で回避する必要がある。
5. MMI 本体とアクセス導波路は別 Shape として挿入される。レイアウト表示上の接続と、製造用マスクのブール演算・DRC 結果は区別する。

10.8　KLayout での確認手順

1. `photonics_pdk_v4.lym` を KLayout の Python マクロとして読み込み、ライブラリ `Photonics` が登録されていることを確認する。
2. ライブラリ内の `MMI_1x2` を配置する。
3. 初期値で矩形本体（x=5〜25 μ m、y=± 2 μ m）と 3 本のアクセス導波路を確認する。
4. 導波路レイヤー `(1,0)` と Pin レイヤー `(2,0)` を表示する。
5. `opt1`、`opt2`、`opt3` のラベルと座標・向きを確認する。
6. `port_spacing` と `mmi_width` を変更し、矩形境界と出力導波路の位置関係を確認する。

10.9　Photonics Router v6.x との連携

MMI は 1 入力 2 出力の分岐点である。Router が光ポートを抽出する際は、**インスタンス変換後のグローバル座標**とポート方向を扱う必要がある。特にミラー・回転配置した MMI では、ローカル座標の ±`d/2` と接続先の上下関係が入れ替わり得る。

また、`opt1` → `opt2`、`opt1` → `opt3` は **2 つの別経路**であり、光ネットリストでは分岐器の 3 ポート接続関係を保つ必要がある。v4 の PCell 単体は分岐比や S パラメータを提供しないため、Router によるネットリスト抽出と光学回路シミュレーションを混同しない。

10.10　v5 への改善案（現行 v4 には未実装）

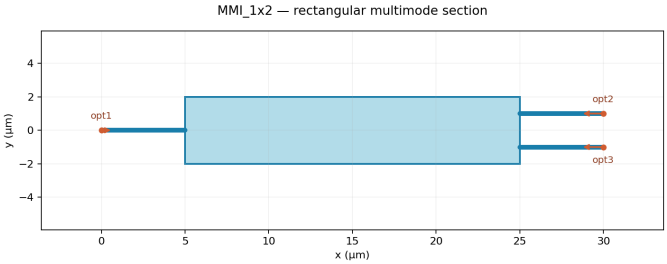
- `coerce_parameters_impl()` 等で寸法の妥当性を検査する。
- 入出力テーパーをパラメータ化し、MMI 本体との遷移を設計できるようにする。
- 波長・偏波・プロセス別の検証済み MMI 寸法をプリセット化する。
- ポートを構造化メタデータとして保持し、Router・ネットリスト抽出に渡す。
- EM シミュレーションで得た S パラメータモデルを関連付ける。
- DRC チェック、最小ギャップ、レイヤー構成を製造プロセスの PDK ルールと連携させる。

10.11　まとめ

`MMI_1x2` は、矩形 MMI 領域、1 本の入力アクセス導波路、2 本の出力アクセス導波路、および 3 つの名前付き光ポートを生成する PCell である。初期形状の全長は 30 µ m で、出力ポート中心は `(30,+1)` と `(30,-1)` µ m に位置する。**この幾何形状は光学特性の保証を意味しない**ため、設計には電磁界解析とプロセス条件に基づく検証が必要である。

実装参照: `pymacros/photonics_pdk_v4.lym`、`class MMI1x2(Base)`、PCell 登録名 `MMI_1x2`。GitHub の `main` は更新される可能性があるため、正式版では参照コミットを固定する。

♣ PCell 構造図（v4 初期パラメータ）



▲図 10.1: 10_mmi_1x2 の中心線・MMI 領域・Optical Port

青：導波路中心線、淡青：*MMI* 領域、橙矢印：`add_pin()` の内向き方向。図は *Python* 実装の座標式に基づく概念図であり、*KLayout* の *GDS* レンダリングではありません。

第 11 章

MZI — マッハ・ツェンダー干渉計

対象実装: `pymacros/photonics_pdk_v4.lym` の `class MZI(Base)` **確認日:** 2026-10-15 **ライブラリ登録名:** `MZI`（Photonics）

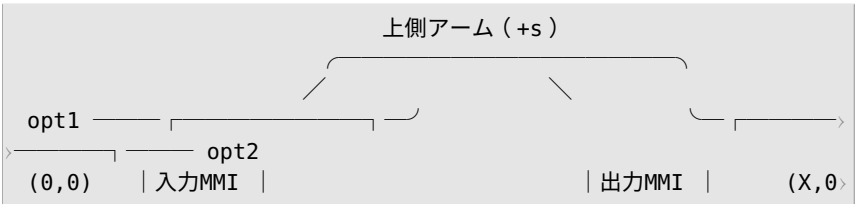
11.1 概要

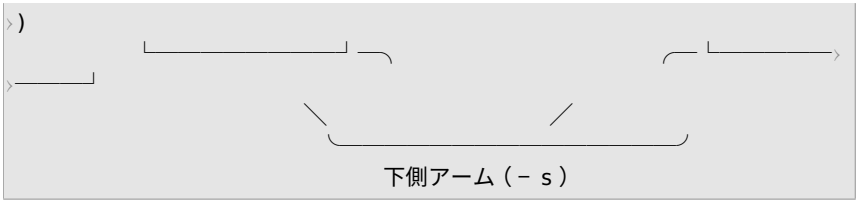
MZI（Mach – Zehnder Interferometer）は、入力光を二つのアームへ分岐し、再合波した光の干渉を利用する素子である。位相変調器、干渉型センサー、光スイッチなどの基本構成に使われる。

v4 の `MZI` は、入力 MMI 矩形、上下 2 本の導波路、出力 MMI 矩形、および入出力の直線導波路を描画する幾何学的 PCell である。 光学伝達関数や損失を自動計算する機能はない。また、現行の MZI PCell にはヒーターや位相シフターのパラメータは存在しない。

11.2 幾何学的構造

▼ text





上図は接続関係を表す模式図であり、寸法比は正確ではない。実装では MMI 本体を `pya.Box`、導波路を `pya.Path` で描画する。MMI からアームへの出発点は `y=±1.0 μ m` に**固定**されており、`arm_spacing` に連動して変わるのは中央アーム位置 `y=± arm_spacing/2` である。

11.3　PCell パラメータ

▼表 11.1: パラメータ・仕様一覧

パラメータ	初期値	単位	意味
`mmi_width`	4.0	μ m	入出力 MMI 矩形の幅
`mmi_length`	20.0	μ m	各 MMI 矩形の長さ
`access_length`	5.0	μ m	入出力の外側直線アクセス長
`arm_length`	80.0	μ m	各アームの中央直線部の X 方向長さ
`arm_spacing`	10.0	μ m	中央直線部の上下アーム中心線間隔
`sbend_length`	20.0	μ m	各 S 字遷移の X 方向長さ
`width`	0.5	μ m	導波路幅
`layer`	1/0	—	導波路レイヤー（共通 Base）
`pin_layer`	2/0	—	光ポートレイヤー（共通 Base）

※ `layer` と `pin_layer` は共通クラス `Base.layers()` で追加される。

11.4　座標の計算

次の記号を用いる。

- `a = access_length`
- `m = mmi_length`
- `b = sbend_length`

- `A = arm_length`
- `s = arm_spacing/2`

入力 MMI は `x ∈ [a,a+m]`、`y ∈ [-mmi_width/2,+mmi_width/2]` の矩形。出力側アームの終点を `x1=a+m+2b+A` とすると、出力 MMI は `x ∈ [x1,x1+m]` である。素子の外側ポート間 X 距離は

$$X = 2a + 2m + 2b + A.$$

初期値を代入すると

$$X = 2(5) + 2(20) + 2(20) + 80 = \boxed{170\ \mu\text{m}}.$$

中央の平行アームは `y=+5 μ m` と `y=-5 μ m`、MMI 接続端は `y=±1 μ m` となる。

11.5 Python 実装の読み方

実装の主要な処理を、ソースの変数名に沿って説明する。

1. `a`, `m`, `s`, `b`, `arm` に各パラメータを代入する。
2. `path_um([(0,0),(a,0)], ...)` で入力アクセス導波路を作る。
3. `pya.Box(...)` で入力 MMI 領域を作る。
4. `bezier(...)` で上側アームの立ち上がりを生成する。
5. 中央直線区間の終点 `(x0+b+arm,s)` を追加する。
6. もう一つの `bezier(...)` で MMI 側の `y=1.0 μ m` へ戻す。
7. `lower=[(x,-y) for x,y in upper]` により、下側アームを上下対称に生成する。
8. 出力 MMI 矩形と出力アクセス導波路を描画する。
9. `add_pin(...)` により `opt1` と `opt2` を定義する。

アームの立ち上がりに使われる制御点は次のとおり。

▼ python

```
upper = bezier(
    (x0, 1.0),
    (x0 + b * .35, 1.0),
    (x0 + b * .65, s),
    (x0 + b, s),
```

```
    48,
)
```

各遷移は 3 次 B é zier 曲線を **48 分割**している。分割数は PCell パラメータとして公開されていない。

11.6　S 字遷移の数学的性質

3 次 B é zier 曲線は

$$\mathbf{B}(t) = (1-t)^3\mathbf{P}_0 + 3(1-t)^2 t\mathbf{P}_1 + 3(1-t)t^2\mathbf{P}_2 + t^3\mathbf{P}_3, \quad 0 \le t \le 1$$

で表される。`P0` と `P1`、および `P2` と `P3` はそれぞれ同じ Y 座標を持つため、両端での接線は水平になる。したがって中央の直線部と**接線方向を揃えられる**。

ただし、接線連続性だけでは曲率連続性は保証されない。曲げ損失を評価するには曲率分布やモード変換を別途確認する必要がある。

11.7　Optical Port の仕様

▼表 11.2: パラメータ・仕様一覧

ポート	位置 (μ m)	inward_angle	接続対象
opt1	(0, 0)	0°	左側入力アクセス導波路
opt2	(X, 0)	180°	右側出力アクセス導波路

初期値では `opt2=(170,0)` μ m である。`add_pin()` はポート位置から内向き方向へ長さ 1 μ m の Pin Path と、同名の Text を `pin_layer` に追加する。

注意: 本 PCell が外部へ公開する光ポートは **2 個のみ**。内部の上下アームと MMI の接合部には、独立した名前付き光ポートはない。

11.8　光学干渉の原理

二つのアームの出力複素振幅を

$$E_1 = A_1 e^{i\phi_1}, \qquad E_2 = A_2 e^{i\phi_2}$$

とすると、同位相で加算する理想化された合波点での強度は

$$I \propto |E_1 + E_2|^2 = A_1^2 + A_2^2 + 2A_1 A_2 \cos(\Delta\phi), \qquad \Delta\phi = \phi_1 - \phi_2.$$

両アームの振幅が等しい場合、合波強度は位相差によって周期的に変わる。実際の MMI を含む MZI の出力透過率は、MMI の散乱行列、位相、損失に依存するため、この式だけでは決定できない。

幾何学的にアーム長が異なると、概略の伝搬位相差は

$$\Delta\phi \simeq \frac{2\pi}{\lambda} n_{\mathrm{eff}} \Delta L$$

である。ただし、屈折率や位相シフターの寄与も含めて考える必要がある。

11.9　v4 の MZI が対称であることの意味

`lower=[(x,-y) for x,y in upper]` としているため、上下の中心線は X 軸に関して鏡映対称である。したがって**幾何学的なアーム中心線長は等しい**。

この PCell には、上下アーム間に意図的な経路長差を与える `delta_length` などのパラメータはない。したがって、v4 の MZI を置いただけでは、意図的な不平衡 MZI を構成できない。位相制御機構も未実装である。

また、導波路の中心線長は X 方向長さ `2b+A` より長い。実長を求める場合は Bézier 点列に沿って線分長を積算する。

$$L_{\mathrm{arm}} \approx \sum_{i=0}^{N-1} \sqrt{(x_{i+1} - x_i)^2 + (y_{i+1} - y_i)^2}.$$

11.10　KLayout での確認手順

1. PDK v4 マクロを読み込み、PCell ライブラリ `Photonics` が登録されていることを確認する。
2. `MZI` を配置し、初期パラメータで 2 つの MMI 矩形と上下対称のアームが表示されることを確認する。

3. レイヤー `1/0` に導波路、`2/0` にポート Path と `opt1`、`opt2` の Text があることを確認する。
4. `arm_spacing` を変更し、中央の平行アームの Y 座標が変わることを確認する。
5. `arm_length` を変更し、中央直線区間と外側ポート間距離が変わることを確認する。
6. ポートの座標と方向が、接続先 PCell との配置変換後にも整合することを確認する。

11.11　Photonics Router v6.x との連携

Router が `pin_layer` 上の Text と Path からポートを抽出する場合、MZI の外部接続点は `opt1`、`opt2` である。インスタンスの回転・反転・移動がある場合は、ローカル座標の位置と方向を**インスタンス変換後のグローバル座標**へ変換する必要がある。

MZI 内部の 2 アームは 1 つの PCell 内部形状として生成されるため、**外部ポート 2 個の情報だけから内部アームの分岐・再合流や各アームの光路長を復元できるとは限らない**。ネットリスト化では、MZI を 1 つのデバイスとして扱うか、内部構造を別途モデル化するかを明確にする必要がある。

11.12　v5 に向けた改善案（未実装）

- `delta_length`、位相シフター長、ヒーターなどを追加して不平衡・可変位相 MZI に対応する。
- MMI 入出力部の接続幅と配置をパラメータ化し、MMI 設計条件を検証する。
- B é zier の離散化数、最小曲率半径、曲率連続性を検査する。
- 各アームの実際の中心線長と差分を計算・表示する。
- 物理レイアウトと光学コンパクトモデルの対応を定義する。
- `arm_spacing` などに対して形状破綻を防ぐ入力値検証を追加する。

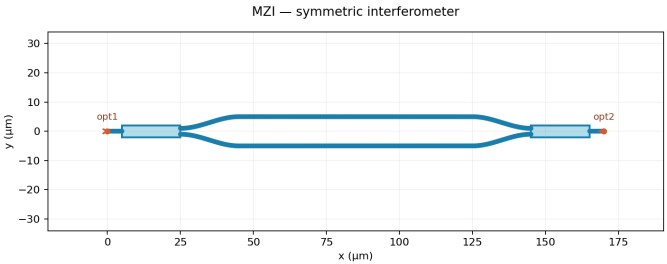
11.13　まとめ

v4 の `MZI` は、2 つの矩形 MMI 領域と上下対称の 2 本のアームを生成する **2 ポート**

のレイアウト PCell である。初期値の全長は 170 µ m。外部ポートは `opt1=(0,0)`、`opt2=(170,0)` である。干渉機能の定量的な検証や位相制御は、今後の光学設計・拡張の対象となる。

参照: `photonics_pdk_v4.lym` (https://github.com/TAKE-HooJoo/klayout-photonics-pdk/blob/main/pymacros/photonics_pdk_v4.lym) の `class MZI(Base)`。**実装との対応:** v4 の公開ソースに記載されたパラメータ・描画処理・ポート定義を基準とする。光学式は説明用の理想モデルである。

♣ PCell 構造図（v4 初期パラメータ）



▲図 11.1: 11_mzi の中心線・MMI 領域・Optical Port

青：導波路中心線、淡青：*MMI* 領域、橙矢印：`add_pin()` の内向き方向。図は *Python* 実装の座標式に基づく概念図であり、*KLayout* の *GDS* レンダリングではありません。

第 12 章

WaveguideRoute — 導波路ルーティング

対象: `pymacros/photonics_pdk_v4.lym` の `WaveguideRoute` PCell **文書状態:** 技術解説ドラフト（2026-10-15）**検証状況:** 2026-10-15 にユーザー提供の `photonics_pdk_v4.lym` を静的解析し、パラメータ・制御点・ポート定義を照合済み。KLayout での実行検証は未実施。

12.1 概要

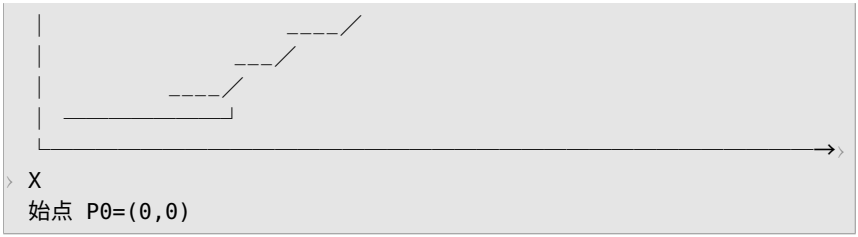
`WaveguideRoute` は、指定した始点と終点の相対位置をもとに、曲線状の光導波路を生成する PCell である。直線導波路や単独の曲げ PCell を組み合わせる代わりに、ひとつの PCell で横方向・縦方向の変位を表現できる。

ただし、**この PCell を配置するだけで、既存の任意の 2 つのインスタンスポートに自動接続されるとは限らない**。始終点の座標・接線方向・ポート幅を、配置先の設計と一致させる必要がある。

12.2 幾何学的な考え方

▼ text

```
Y
↑                                  終点 P3=(dx,dy)
|                                ┌──────────→
```



図は曲線の概念図であり、実装の制御点配置や実際の曲率分布を厳密に示すものではない。

局所座標系の始点を `P0=(0,0)`、終点を `P3=(dx,dy)` とすると、両端で水平接線を持つ 3 次 Bézier 曲線は、たとえば次の 4 制御点で構成できる。

$$P_0 = (0,0), \quad P_1 = (c_0, 0), \quad P_2 = (dx - c_1, dy), \quad P_3 = (dx, dy).$$

ここで `c0` と `c1` は制御距離である。v4 では `c0=c1=sign*ctrl` としており、具体的な式は 12.7 節に示す。

12.3　PCell パラメータ

ユーザー提供の v4 ソース内の `param(...)` と照合した初期値は次のとおりである。

▼表 12.1: パラメータ・仕様一覧

パラメータ	記録された初期値	単位	役割
`dx`	100	μm	始点から終点までの X 方向変位
`dy`	20	μm	始点から終点までの Y 方向変位
`width`	0.5	μm	導波路幅
`radius`	10	μm	制御点距離を決めるための設計パラメータ
`points`	24	—	曲線の離散化点数に関するパラメータ

共通の `layer` / `pin_layer` は `Base.layers()` で定義され、それぞれ `pya.LayerInfo(1,0)` / `pya.LayerInfo(2,0)` を初期値とする。

注意： `radius` は必ずしも曲線上の最小曲率半径ではない。曲率制約を保証するパラメータとして扱ってはいけない。

12.4　3 次 B é　zier 曲線

3 次 B é　zier 曲線は、パラメータ `t ∈ [0,1]` に対して

$$\mathbf{B}(t) = (1-t)^3\mathbf{P}_0 + 3(1-t)^2 t\mathbf{P}_1 + 3(1-t)t^2\mathbf{P}_2 + t^3\mathbf{P}_3$$

で定義される。接線ベクトルは

$$\mathbf{B}'(t) = 3(1-t)^2(\mathbf{P}_1-\mathbf{P}_0) + 6(1-t)t(\mathbf{P}_2-\mathbf{P}_1) + 3t^2(\mathbf{P}_3-\mathbf{P}_2)$$

となる。端点では

$$\mathbf{B}'(0) = 3(\mathbf{P}_1-\mathbf{P}_0), \qquad \mathbf{B}'(1) = 3(\mathbf{P}_3-\mathbf{P}_2)$$

である。したがって、始点側の 2 制御点と終点側の 2 制御点をそれぞれ水平方向に並べれば、両端で水平な接線を設計できる。ただし制御距離がゼロの場合には接線ベクトルが退化することがある。

12.5　曲率と最小曲率半径

平面曲線 `B(t)=(x(t),y(t))` の曲率は

$$\kappa(t) = \frac{|x'(t)y''(t) - y'(t)x''(t)|}{\left(x'(t)^2 + y'(t)^2\right)^{3/2}}$$

である。曲率半径は `R(t)=1/κ (t)`（`κ≠ 0`）で求める。

`radius=10 μ m` と入力しただけでは、全区間で `R(t) ≥ 10 μ m` は保証されない。 設計ルールとして最小曲率半径が必要な場合は、生成された実際の中心線を数値的に検査する。

曲線端と直線の接続についても、接線方向が一致する **G1 連続** と、曲率まで連続する **G2 連続** は区別する必要がある。

12.6　導波路長

理想的な連続曲線の中心線長は

$$L = \int_0^1 \sqrt{x'(t)^2 + y'(t)^2}\, dt$$

である。KLayout のレイアウトでは有限個の点を結ぶ多角線として近似することが多く、点列 `Q0,…,QN` から

$$L_{\mathrm{poly}} = \sum_{i=0}^{N-1} \|Q_{i+1} - Q_i\|$$

を計算できる。点数、DBU 丸め、曲線の形状によって近似誤差が変化する。光学位相計算に用いる長さは、中心線長と実効屈折率の定義を揃える必要がある。

12.7　v4 の Python 実装（ソース照合済み）

制御点距離は以下の式で決定される。

▼ python

```
ctrl = max(abs(dx)*0.35, min(abs(dx)/2.0, max(self.radius, 1.0)
))
sign = 1 if dx >= 0 else -1
p0 = (0.0, 0.0)
p3 = (dx, dy)
p1 = (sign*ctrl, 0.0)
p2 = (dx-sign*ctrl, dy)
pts = bezier(p0, p1, p2, p3, max(32, int(self.points)*2))
```

`points` の値は直接の点数ではなく、サンプリング分割数 `n=max(32,2*points)` を決める。共通 `bezier` 関数は `n+1` 点を返す。初期値 `points=24` では **49 点** となる。`radius` は制御点距離の決定に使用されるが、最小曲率半径を保証しない。

`dx<0` では `sign=-1` に切り替わる。`dx=0` では制御点が一致し得るため、ゼロ変位や退化形状は KLayout で追加検証が必要である。導波路は `path_um` により整数座標の `pya.Path` に変換される。

12.8　Optical Port（ソース照合済み）

`add_pin()` は Pin レイヤーに内向きの短い Path と名前の Text を挿入する。`dx=0` でもコード上は `dx>=0` 側の角度となる点に注意する。

▼表 12.2: パラメータ・仕様一覧

条件	opt1 位置・内向き方向	opt2 位置・内向き方向
`dx >= 0`	`(0,0)`, 0°	`(dx,dy)`, 180°
`dx < 0`	`(0,0)`, 180°	`(dx,dy)`, 0°

12.9 KLayout での確認手順

1. PCell Library v4 をロードし、`WaveguideRoute` を配置する。
2. `dx=100 μ m`、`dy=20 μ m`、`width=0.5 μ m` を基準として形状を確認する。
3. `dy=0` にして直線相当の形状になるか確認する。
4. `dy` の符号を反転し、曲線の上下対称性を確認する。
5. `radius` を変化させ、曲線形状・最小曲率半径・導波路長を比較する。
6. `points` を増減し、形状の滑らかさと生成時間を比較する。
7. Pin レイヤーを表示し、端点の座標と向きを確認する。

12.10 Photonics Router v6.x との関係

`WaveguideRoute` は**指定されたパラメータから 1 本の経路形状を生成する PCell** であり、Photonics Router は**複数のインスタンス・光ポートを対象に接続経路を設計するツール**として役割を区別できる。

▼表 12.3: パラメータ・仕様一覧

観点	WaveguideRoute PCell	Photonics Router v6.x
主な入力	PCell の相対寸法・曲線パラメータ	接続対象の光ポート・配置情報
主な出力	1 つの導波路形状	接続経路、関連するポート情報など
形状生成	PCell 再生成で更新	Router の経路生成処理に依存
自動障害物回避	実装確認が必要	Router のバージョン・機能に依存
接続ネットリスト	PCell 単体では通常対象外	v6.x の抽出機能と連携可能

両者を連携させる場合は、ポート角度の定義、中心線長の算出方法、セル変換（回転・反転）時の座標系を統一することが重要である。

12.11 PCell Library v5 への改善提案

- **曲率制約:** 最小曲率半径を数値検証し、条件違反を警告する。
- **接線制御:** 始点・終点の方向角を明示的なパラメータとして追加する。
- **ポート仕様統一:** Pin の位置・方向・幅・名称を共通 API で生成する。
- **長さ情報:** 曲線中心線長を PCell のメタデータとして保存する。
- **入力検証:** `points`、`width`、`dx` などの不適切な値を検出する。
- **経路品質:** B é zier、Euler、円弧などの選択肢を比較可能にする。

これらは **v4 に実装済みという意味ではなく、将来版への提案**である。

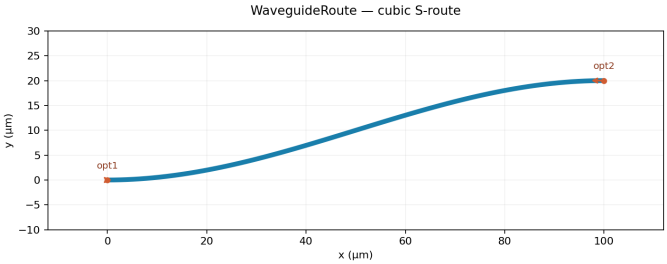
12.12　まとめ

`WaveguideRoute` は、相対変位に基づく導波路経路を PCell として再利用できる点に価値がある。一方、曲線の制御パラメータ、光ポート方向、最小曲率半径、導波路長は別々に検証する必要がある。

公開前の必須照合: v4 の `WaveguideRoute` クラスにおける制御点式、パラメータ定義、`add_pin(...)`、形状生成 API を原本ソースと比較すること。

参 照:　Photonics PDK v4 ソースコード (https://github.com/TAKE-HooJoo/klayout-photonics-pdk/blob/main/pymacros/photonics_pdk_v4.lym)
前章: [第 11 章 MZI](11_mzi.md) **次章:** 第 13 章 PortConnector（制作予定）

♣ PCell 構造図（v4 初期パラメータ）



▲図 12.1: 12_waveguide_route の中心線・MMI 領域・Optical Port

青：導波路中心線、淡青：*MMI* 領域、橙矢印：***add_pin()*** の内向き方向。図は *Python* 実装の座標式に基づく概念図であり、*KLayout* の *GDS* レンダリングではありません。

第 13 章

PortConnector — 光ポート接続・位置合わせ

対象ソース: `pymacros/photonics_pdk_v4.lym`、`PortConnector` クラス（v4）
文書状態: ソースコード照合済み（2026-10-15）**参照:** https://github.com/TAKE-HooJoo/klayout-photonics-pdk/blob/main/pymacros/photonics_pdk_v4.lym

13.1 概要

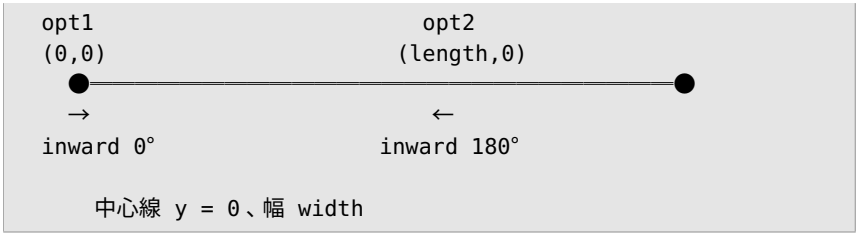
`PortConnector` は、**短い直線導波路と、名前付きの 2 つの光ポート**を生成する位置合わせ用 PCell である。光ポートの可視化、接続位置の確認、Router の動作試験などに利用できる。

v4 の実装では `Straight` と幾何形状・ポート生成処理がほぼ同じであり、特別な自動スナップ機構や、2 つの既存 PCell インスタンスを探索して接続する機能は持たない。「Connector」という名称は**接続補助用の短い直線 PCell** という意味で理解する。

13.2 形状と座標

▼ text

```
                +X →
```



- 中心線：(0, 0) から (length, 0)
- 導波路幅：width
- 光ポート：左端 opt1、右端 opt2
- デフォルトでは length=10.0 μ m、width=0.5 μ m

形状は波導レイヤー上の pya.Path として生成される。

13.3 PCell パラメータ

▼表 13.1: パラメータ・仕様一覧

パラメータ	型	初期値	単位	役割
length	TypeDouble	10.0	μ m	接続補助導波路の中心線長
width	TypeDouble	0.5	μ m	導波路幅
layer	TypeLayer	(1, 0)	—	導波路の描画レイヤー
pin_layer	TypeLayer	(2, 0)	—	光ポートの描画レイヤー

layer と pin_layer は Base.layers() によって共通定義される。

13.4 v4 の実装

次の抜粋は PortConnector クラスの実装である。

▼python

```
class PortConnector(Base):
    """Visible alignment helper: a short waveguide with two nam›
›ed optical ports."""
    def __init__(self):
```

```
        super(PortConnector,self).__init__()
        self.param("length",self.TypeDouble,"Connector length (
um)",default=10.0)
        self.param("width",self.TypeDouble,"Width (um)",default
=0.5)
        self.layers()

    def produce_impl(self):
        l,c=self.layout,self.cell; wg=l.layer(self.layer); pin=
l.layer(self.pin_layer)
        c.shapes(wg).insert(path_um([(0,0),(self.length,0)],sel
f.width,l.dbu))
        add_pin(c,l,pin,"opt1",0,0,0,self.width)
        add_pin(c,l,pin,"opt2",self.length,0,180,self.width)
```

♣ 処理の流れ

1. `__init__()` で長さ・幅・共通レイヤーを宣言する。
2. `produce_impl()` で導波路レイヤーと Pin レイヤーを取得する。
3. `path_um()` で中心線を KLayout の DBU 整数座標へ変換し、`pya.Path` を挿入する。
4. `add_pin()` で両端に光ポートを作成する。

13.5 Optical Port の仕様

▼表 13.2: パラメータ・仕様一覧

名前	ローカル位置（μ m）	内向き角度	ポート幅
opt1	(0, 0)	0°（ +X）	width
opt2	(length, 0)	180°（− X）	width

`add_pin()` は Pin レイヤーに**短い Path とポート名の Text** を挿入する。既定の Pin Path 長は `1.0 μ m` であり、端点から PCell 内部へ向けて描かれる。Pin Path は導波路レイヤーとは別レイヤーに置かれる。

注意： 角度はポートの**内向き**方向を示す。外部の接続先へ向かう方向として扱う場合は 180°反転が必要である。 Router 側のポート抽出仕様と必ず照合する。

13.6　Straight との比較

▼表 13.3: パラメータ・仕様一覧

項目	Straight	PortConnector
初期長さ	100 µ m	10 µ m
初期幅	0.5 µ m	0.5 µ m
中心線形状	直線	直線
光ポート	opt1, opt2	opt1, opt2
ポート角度	0° / 180°	0° / 180°
主な想定用途	一般の直線導波路	接続位置合わせ・テスト

実装上の主な違いは初期長さと用途を表す名称である。 PortConnector 固有の接続判定アルゴリズムはない。

13.7　導波路長と光学位相

導波路中心線長は単純に

$$L = \texttt{length}$$

である。単一モード・一様断面の近似では、波長 \(\lambda\)、実効屈折率 \(n_\mathrm{eff}\) に対する位相遅延は

$$\phi = \beta L = \frac{2\pi n_{\mathrm{eff}}}{\lambda} L$$

となる。これは光学設計上の理論式であり、PCell 自体は \(n_\mathrm{eff}\) や位相を計算しない。

13.8　KLayout での確認手順

1. photonics_pdk_v4.lym を KLayout で読み込み、Photonics ライブラリが登録されていることを確認する。
2. PCell 一覧から PortConnector を選択する。
3. length=10.0、width=0.5 のまま配置する。

4. レイヤー `(1,0)` に長さ 10 µ m の導波路が描画されていることを確認する。
5. レイヤー `(2,0)` を表示し、両端の `opt1` / `opt2` のラベルと短い Pin Path を確認する。
6. 長さを変更し、右側の `opt2` が指定した X 座標へ移動することを確認する。

13.9 Photonics Router v6.x との連携

`PortConnector` は、ポート抽出機能のテストに適した単純な 2 ポート素子である。たとえば、2 つの `PortConnector` インスタンスを離して配置し、Router がポートの座標・方向・幅・インスタンス変換を正しく解釈できるか検証できる。

ただし、以下の点は **Router 側の実装依存**である。

- Pin レイヤー上の Text と Path からのポート認識方法
- 回転・反転したインスタンスのポート座標変換
- 内向き角度とルーティング出発方向の対応
- 接続後の導波路長計算とネットリスト抽出

`PortConnector` を配置するだけで自動接続やネットリスト生成が実行されるわけではない。

13.10 設計上の注意点

- **長さ・幅の検証:** v4 のこの PCell には、負値やゼロ値を明示的に拒否する入力チェックがない。運用上は `length > 0`、`width > 0` を前提とする。
- **DBU 丸め:** `q(v, dbu) = int(round(v / dbu))` により座標・幅が整数 DBU に丸められる。非常に小さい寸法では丸め誤差に注意する。
- **レイヤーの分離:** Pin Path は光導波路本体の形状ではない。描画や検証で Pin レイヤーを導波路と混同しない。
- **ポートの向き:** 左端は +X、右端は− X が「内向き」であり、導波路の伝搬方向や外向き法線とは必ずしも同じではない。
- **自動接続ではない:** インスタンス間の幾何学的接続や衝突回避は別途 Router 等で行う。

13.11　v5 への改善案（未実装）

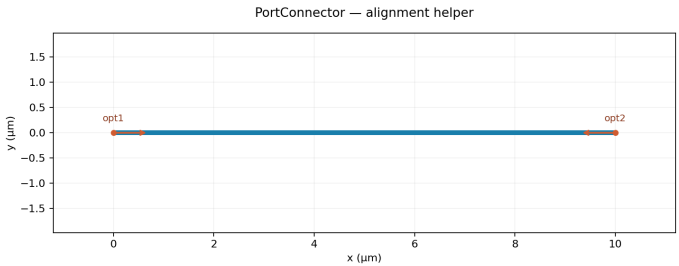
1. `length`、`width` に対する正値チェックを追加する。
2. ポートの中心位置・方向・幅を、テキストと Pin Path に加えて構造化メタデータとして扱う。
3. `Straight` と重複する実装を共通クラスまたはユーティリティへ整理する。
4. 接続許容誤差、幅の一致、ポート方向の整合性を検証する機能を Router 側と協調して追加する。
5. 回転・ミラー・階層インスタンスを含む自動テストを整備する。

これらは提案であり、v4 で実装済みの機能ではない。

13.12　まとめ

`PortConnector` は、**短い直線導波路＋ 2 つの名前付き光ポート**という最小構成の PCell である。初期長さ 10 µ m で、光ポートの表示、位置合わせ、Photonics Router のポート抽出試験に適している。一方、v4 には自動スナップや自動接続機能はない。設計時はポートの内向き角度、Pin レイヤー、DBU 丸めを正しく理解して利用する。

♣ PCell 構造図（v4 初期パラメータ）



▲図 13.1: 13_port_connector の中心線・MMI 領域・Optical Port

青：導波路中心線、淡青：*MMI* 領域、橙矢印：`add_pin()` の内向き方向。図は *Python* 実装の座標式に基づく概念図であり、*KLayout* の *GDS* レンダリングではありません。

第 14 章

Optical Port の共通仕様

対象: KLayout Silicon Photonics PDK v4、10 種類の PCell、Photonics Router v6.x **文書状態:** 技術解説ドラフト（2026-10-15）**参照ソース:** `pymacros/photonics_pdk_v4.lym` (https://github.com/TAKE-HooJoo/klayout-photonics-pdk/blob/main/pymacros/photonics_pdk_v4.lym)

照合上の注意：本章は第 1〜13 章の既存資料を基礎としている。今回、GitHub 上の現行マクロを再取得できなかったため、`add_pin()` の正確な引数順・Text 属性・Router の抽出ロジックは、公開前に該当コミットのソースと照合すること。説明用コードは実装例であり、v4 からの逐語引用ではない。

14.1 Optical Port とは

光ポート（Optical Port）は、光導波路や光学素子の接続位置と接続方向を表すインターフェースである。レイアウト設計では少なくとも次の情報を区別する。

▼表 14.1: パラメータ・仕様一覧

属性	意味	例
名前	素子内での識別子	`opt1`, `opt2`
中心座標	導波路中心線と接続境界の交点	`(0, 0)` μ m
方向	ポートの接続軸の向き	0°, 90°, 180°, 270°
幅	接続する導波路の幅	0.5 μ m
レイヤー	ポートを表現する図形の格納先	`2/0`

光ポートは物理的な光学特性そのものを保証しない。 ポート位置が一致しても、幅・モード・偏波・曲率・製造条件が不適切なら、接続損失が発生し得る。

14.2　v4 のレイヤー表現

既存の v4 解説では、導波路本体とポートマーカーを別レイヤーに描く構成として整理している。

▼表 14.2: パラメータ・仕様一覧

レイヤー	既定値	用途
Waveguide	`(1, 0)`	導波路の幾何形状
Optical Pin	`(2, 0)`	ポート位置・方向・名前の表示

Pin レイヤー上には短い `Path` と名前を示す `Text` を配置する。`Path` の中心線がポートの向きを表し、`Text` が `opt1` などの名前を示す。この**図形ベースの表現**と、Router 内部の**構造化されたポート情報**は別の概念である。

▼text

```
Waveguide layer (1/0)       Pin layer (2/0)

 opt1             opt2       opt1             opt2
   ●──────────────────────────●          →                ←
      光導波路                    短いPath + Text
```

14.3　ローカル座標と内向き方向

PCell の座標は、通常、**PCell 自身のローカル座標系**で定義する。たとえば `Straight(length=100)` は、左端を `(0, 0)`、右端を `(100, 0)` μ m とする。

v4 の既存章では、ポート方向を**素子内部へ向かう角度（内向き角度）**として説明している。

▼表 14.3: パラメータ・仕様一覧

位置	内向き方向	外部へ向かう方向
左端	0°（ +X）	180°（− X）
右端	180°（− X）	0°（ +X）
下端	90°（ +Y）	270°（− Y）
上端	270°（− Y）	90°（ +Y）

重要： Pin Path が「ポートから内部へ伸びる」方式なら、Path の向きは内向きである。Router で接続経路の開始方向を求める場合、内向き・外向きの取り扱いを明示し、混同しないこと。

角度の正規化は、

$$\theta_{\mathrm{norm}} = (\theta \bmod 360^\circ)$$

外向きへの変換は、

$$\theta_{\mathrm{out}} = (\theta_{\mathrm{in}} + 180^\circ) \bmod 360^\circ$$

で表せる。

14.4　PCell インスタンスの回転と反転

KLayout では、同じ PCell を回転・反転して配置できる。**ポート抽出には、ローカル座標からトップセル座標への変換が必要**である。

回転角を ϕ、平行移動を (t_x, t_y) とすると、反転を伴わない場合は

$$\begin{pmatrix} x' \\ y' \end{pmatrix} = \begin{pmatrix} \cos\phi & -\sin\phi \\ \sin\phi & \cos\phi \end{pmatrix} \begin{pmatrix} x \\ y \end{pmatrix} + \begin{pmatrix} t_x \\ t_y \end{pmatrix}.$$

方向ベクトル $d = (\cos\theta, \sin\theta)$ も、同じ線形変換で回転する。ただし、**平行移動は方向に作用しない**。

鏡映を含む場合、角度に単純に回転角を加えるだけでは不十分である。変換行列 A を使い、

$$\mathbf{d}' = A\mathbf{d}, \qquad \theta' = \mathrm{atan2}(d'_y, d'_x)$$

と計算する。これにより、回転と鏡映を統一的に扱える。

♣ 数値例

ローカル座標 `(100, 0)` µ m、内向き 180°の右端ポートを、原点周りに 90°回転して `(20, 30)` µ m 平行移動した場合：

- 変換後の位置：`(20, 130)` µ m
- 変換後の内向き方向：270°
- 変換後の外向き方向：90°

14.5　10 種類の PCell とポート数

既存の v4 解説に記載されたポート数を整理する。

▼表 14.4: パラメータ・仕様一覧

PCell	ポート数	接続上の役割
`Straight`	2	直線接続
`Bend90`	2	90°方向転換
`EulerBend`	2	曲率を考慮した方向転換
`SBend`	2	横方向オフセット
`Ring`	2	バス導波路の入出力（リング自体のポートではない）
`DirectionalCoupler`	4	2 入力・2 出力の結合器
`MMI_1x2`	3	1 入力・2 出力の分岐器
`MZI`	2	干渉素子の入出力
`WaveguideRoute`	2	指定経路の始点・終点
`PortConnector`	2	ポート確認・接続補助

ポートの**名前・位置・角度**は、各 PCell の章に記載した値を参照する。`Ring` の 2 ポートは単一バス型構成に対応する。

14.6　add_pin() の役割

既存の v4 資料では、共通関数 `add_pin()` が光ポートを生成すると整理している。一般的な処理は以下のとおり。

1. ポート中心の座標をμ m から DBU 整数座標へ変換する。
2. 指定方向に短い Pin Path を描く。
3. ポート名を Text として Pin レイヤーに描く。
4. 導波路の幅に対応する Pin Path 幅を設定する。

既存の第 13 章には次の**呼び出し形式**が記録されている。

▼python

```
add_pin(c, l, pin, "opt1", 0, 0, 0, self.width)
add_pin(c, l, pin, "opt2", self.length, 0, 180, self.width)
```

これは**呼び出し例**であり、`add_pin()` 関数本体の実装詳細は現行ソースと照合する必要がある。

♣ DBU 変換の例

レイアウトの DBU が `0.001 µ m` の場合、`10.0 µ m` は `10000 DBU` に相当する。

$$N_{\mathrm{DBU}} = \mathrm{round}\left(\frac{x_{\mu\mathrm{m}}}{\mathrm{DBU}}\right)$$

整数化による位置誤差は、最近傍丸めなら通常最大 `0.5 DBU` 程度である。ただし、個々の座標変換や丸め方の実装によって誤差特性は変わる。

14.7　ポート接続の成立条件

2 つのポート A、B を接続する場合、少なくとも以下を確認する。

位置の一致

$$\|\mathbf{p}_A - \mathbf{p}_B\| \le \varepsilon_p$$

方向の整合（両方が内向き角度の場合）

$$|\mathrm{wrap}_{180}(\theta_A - \theta_B - 180^\circ)| \le \varepsilon_\theta$$

幅の整合

$$|w_A - w_B| \le \varepsilon_w$$

ここで、`ε p`、`εθ`、`ε w` は **Router 側で設計する許容値**であり、v4 に実装済みの既定値を意味しない。

♣ 位置が一致していない場合

接続ルートを生成するときは、単に始点と終点を結ぶだけでなく、**ポートの接線方向**と**最小曲率半径**も考慮する。ポートが対向していても、障害物や曲率制約により接続できない場合がある。

14.8　Router v6.x との連携

Photonics Router 側の設計では、以下の処理を明確に分離するとよい。

▼text

```
PCellインスタンス
        ↓
Pin Path / Textの抽出
        ↓
ポート名・中心・方向・幅の解析
        ↓
インスタンス変換の適用
        ↓
トップセル座標でのポート一覧
        ↓
接続候補の判定
        ↓
経路生成・導波路長計算
        ↓
ネットリスト抽出・検証
```

上記は**推奨する論理構成**であり、Router v6.x の内部実装を逐語的に記述したものではない。

特に注意したいのは、**ポート抽出結果とネットリストの整合性**である。接続の識別に名前だけを使うと、複数インスタンスの `opt1` が衝突する。ネットリストでは、たとえば `(instance_id, port_name)` の組を識別子として扱うことが望ましい。

14.9　実装検証チェックリスト

♣ 推奨テストケース

1. `Straight` を 0°・ 90°・ 180°・ 270°で配置し、 2 ポートの座標と角度を比較する。
2. `Straight` を鏡映し、内向き・外向き角度の変換を確認する。
3. `DirectionalCoupler` の 4 ポートを抽出し、上下・左右の対応を確認する。
4. `MMI_1x2` の 3 ポートが重複なく抽出されることを確認する。
5. 同じ PCell を複数配置し、`opt1` の名前衝突が起きないことを確認する。
6. PCell のパラメータを変更し、ポート座標が追随することを確認する。

▼表 14.5: パラメータ・仕様一覧

項目	確認内容
ポート名	同一インスタンス内で重複しない
Pin レイヤー	全 PCell で設定が統一されている
ポート位置	導波路中心線の端点と一致する
方向	内向き／外向きの定義が一致する
回転	90°・ 180°・ 270°で座標と角度が正しい
反転	鏡映後の方向ベクトルが正しい
DBU	μ m から DBU への変換が一貫している
幅	接続する導波路幅と整合する
階層	ネストしたインスタンスでもトップ座標に変換できる
ネットリスト	インスタンス識別子とポート名で一意に識別できる

14.10 PCell Library v5 への改善案

提案 1：構造化されたポート仕様図形マーカーとは別に、`name`、`position`、`orientation`、`width`、`layer` などを明示的なデータとして取り扱う。

提案 2：方向の統一 `orientation` が内向きか外向きかを API 仕様で固定し、Router 側との変換規則を明文化する。

提案 3：接続検証 API 位置・方向・幅・レイヤーの一致を確認する共通関数を提供する。

提案 4：階層・鏡映テストトップセルに配置した複数インスタンスについて、回転・反転・階層変換を自動テストする。

提案 5：ネットリストとの一貫性ポート識別子、接続関係、導波路長の算出元を統一し、検証可能なデータ形式にする。

14.11 まとめ

Optical Port は、PCell と Photonics Router を結ぶ重要な共通インターフェースである。v4 では Pin レイヤー上の Path と Text を利用することで、レイアウト上でポート位置と名前を可視化できる。一方、回転・反転・階層変換、方向の定義、ポート幅、インスタンス識別などを統一して扱うことが、安定した自動配線とネットリスト抽出には欠かせない。

次章：第 15 章「Photonics Router との連携」

第 15 章

Photonics Router v6.x との連携

対象: KLayout Silicon Photonics PDK v4 / Photonics Router v6.x **文書状態:** 技術解説ドラフト（2026-10-15）**関連章:** [第 12 章 WaveguideRoute](12_waveguide_route.md)、[第 14 章 Optical Port の共通仕様](14_optical_ports.md)

実装照合に関する注意：本章は既存の PCell 解説および開発履歴（Router v6.0 で光ネットリスト抽出を追加、v6.2 系を試作）をもとに、連携設計を体系化したものです。Router 各版の現行 Python コードをこの章の制作時に直接照合していません。以下の疑似コード、データ構造、検証手順は**推奨設計・検証例**であり、現行版にすべて実装済みという意味ではありません。

15.1 本章の目的

PCell Library v4 は、導波路・分岐器・干渉計などの**パラメトリックな素子形状**を生成する。一方、Photonics Router は、レイアウト上の光ポートを起点として**素子間の導波路接続**を構築・評価する役割を担う。

両者の連携では、単に導波路図形を接触させるだけでなく、次を整合させる必要がある。

- 接続位置と接線方向
- 導波路幅・レイヤー・DBU
- 曲率および最小曲率半径
- ポート識別子と階層インスタンス

- 接続経路の長さとネットリスト

15.2　システム全体の構成

▼text

```
PCell Library v4
  Straight / Bend90 / EulerBend / SBend / Ring / ...
        │
        ├──── Waveguide形状（例：1/0）
        └──── Pin Path + Text（例：2/0）
                    │
                    ▼
        ポート抽出・正規化
        （階層・座標・方向・幅）
                    │
                    ▼
        接続元／接続先の選択
                    │
                    ▼
        Photonics Router
        （経路候補・曲率・障害物判定）
                    │
                    ▼
        導波路図形の生成・配置
                    │
                    ▼
        長さ計算・接続検証
                    │
                    ▼
        光ネットリスト抽出・出力
```

この流れは**推奨する論理アーキテクチャ**であり、各 Router 版の実際の UI 操作順序と同一とは限らない。

15.3　Optical Port 抽出

♣ 入力情報

PCell v4 では、既存の解説によれば Pin レイヤー上の短い Path と Text を使ってポートを表現する。抽出側は少なくとも次を取得する。

▼表 15.1: パラメータ・仕様一覧

項目	内容	留意点
`instance_path`	階層内のインスタンス識別	同名ポートの衝突を回避
`port_name`	`opt1` など	セル内では一意であることを確認
`position`	トップセル上の座標	DBU を統一
`direction`	接続軸の向き	内向き／外向きを区別
`width`	接続導波路幅	幾何学的幅と照合
`layer`	光導波路・Pin のレイヤー	レイヤーマッピングに注意

`opt1` だけをグローバルなキーにしてはいけない。たとえば `U1/opt1` と `U2/opt1` は別ポートである。

♣ 階層座標への変換

ローカル座標のポート位置を \(\mathbf p\)、インスタンスの線形変換を \(A\)、平行移動を \(\mathbf t\) とすると、

$$\mathbf{p}_{\mathrm{top}} = A\mathbf{p} + \mathbf{t}$$

方向ベクトル \(\mathbf d\) は、

$$\mathbf{d}_{\mathrm{top}} = A\mathbf{d}$$

で変換する。**方向に平行移動を加えない**。鏡映がある場合も、角度値ではなく方向ベクトルを変換すると安全である。

♣ 内向き／外向きの規約

第 14 章で説明した v4 の内向き方向を使用する場合、Router がポートから外へ進む初期接線は、

$$\theta_{\mathrm{route}} = (\theta_{\mathrm{in}} + 180^\circ) \bmod 360^\circ$$

とする。ただし、**実際の Pin Path から方向を推定する方式**では、Path の端点順序や向きがコードの規約と一致するか確認する必要がある。

15.4　ポート間接続の成立条件

接続する 2 つのポートについて、以下を確認する。

1. **位置:** ルート端点とポート中心が許容誤差内で一致する。
2. **方向:** ルートの接線がポートの外向き方向と整合する。
3. **幅:** 幅が一致するか、テーパーなどの明示的な変換素子を使用する。
4. **レイヤー:** 接続可能な光導波路レイヤーに属する。
5. **形状:** 接続部に隙間・重複・急峻な折れ曲がりがない。

位置誤差の例：

$$\|\mathbf{p}_{\text{route}} - \mathbf{p}_{\text{port}}\| \leq \varepsilon_p$$

角度誤差の例：

$$|\text{wrap}_{180}(\theta_{\text{route}} - \theta_{\text{expected}})| \leq \varepsilon_\theta$$

ε　p、εθ　は**推奨検証パラメータ**であり、v4/v6.x の確定済み初期値ではない。

15.5　導波路ルーティング方式

♣ WaveguideRoute PCell との違い

▼表 15.2: パラメータ・仕様一覧

項目	WaveguideRoute（PCell）	Photonics Router（ツール）
主な入力	PCell パラメータ	接続元・接続先ポート等
主な出力	1 つの導波路形状	接続経路の生成・管理
ポート探索	PCell 単体の責務外	Router 側の機能領域
障害物回避	自動回避を前提としない	機能・版ごとに確認が必要
ネットリスト	単体では抽出しない	v6.0 で抽出機能を追加した開発履歴あり

♣ 曲線の候補

- 円弧＋直線：半径と形状の理解が容易。
- Euler 曲線：曲率変化を滑らかに設計しやすい。
- B é zier 曲線：端点位置・接線を制御しやすい。
- CSC / CSCSC 型：直線（S）と曲線（C）を組み合わせる経路表現。v6.2 系の試作履歴がある。

注意: ある方式が実装されているか、障害物回避・曲率保証まで実現しているかは、Router の版とソースコードで個別に確認する。

15.6　曲率制約と幾何学的検証

導波路中心線を \((x(t),y(t))\) とすると曲率は、

$$\kappa(t) = \frac{|x'(t)y''(t) - y'(t)x''(t)|}{(x'(t)^2 + y'(t)^2)^{3/2}}$$

曲率半径は \(R(t)=1/\kappa(t)\)（\(\kappa\ne0\)）。指定した設計最小半径 \(R_{\min}\) に対して、

$$\max_t \kappa(t) \le \frac{1}{R_{\min}}$$

を確認する。Bezier の制御点から推測するだけでなく、生成した中心線をサンプリングして評価することが重要である。折れ線化した Path の各頂点の角度変化も別途点検する。

15.7　導波路長の計算

連続曲線の幾何学的長さは、

$$L = \int_{t_0}^{t_1} \sqrt{(x'(t))^2 + (y'(t))^2}\, dt$$

離散化後の近似長は、

$$L_{\mathrm{poly}} = \sum_{i=0}^{N-2} \sqrt{(x_{i+1}-x_i)^2 + (y_{i+1}-y_i)^2}$$

で求める。サンプリング数・DBU 丸め・曲線近似によって値が変わるため、**理論上の中心線長と GDS 上の近似長**を区別して記録する。

光学位相を概算する場合は、

$$\phi = \frac{2\pi}{\lambda} \int n_{\mathrm{eff}}(s, \lambda)\, ds$$

と表せるが、長さだけから実際の位相・損失が確定するわけではない。

15.8　光ネットリスト抽出

開発履歴上、Photonics Router v6.0 では光ネットリスト抽出が追加された。一般的なネットリストでは、**素子インスタンスをノード、光ポート間の接続をエッジ**として表現できる。

♣ 推奨する識別モデル

▼ text

```
Component: U1 (MMI_1x2)
  U1/opt1
  U1/opt2
  U1/opt3

Component: U2 (Straight)
  U2/opt1
  U2/opt2

Connection:
  U1/opt2 -- route_001 -- U2/opt1
```

これは**概念的な例**であり、Router v6.0 の実際の出力フォーマットではない。

検証では次を確認する。

- 同じ接続が重複登録されない。
- 未接続ポートが意図せず消えない。
- 接続方向の逆転で接続が欠落しない。
- 階層内の同名ポートが混同されない。
- ルートの形状を更新した後、ネットリストも更新される。

15.9　KLayout での検証シナリオ

▼表 15.3: パラメータ・仕様一覧

試験	配置・操作	期待する確認結果
T01	Straight 2 個を水平配置	端点と方向を正しく抽出
T02	片方を 180°回転	内向き・外向きの変換が一致
T03	片方を鏡映	鏡映後の角度が正しい
T04	MMI_1x2 と Straight を接続	`opt1` などの同名ポートを区別
T05	DirectionalCoupler の 4 ポート	全ポートを一意に抽出
T06	幅の異なる導波路を接続	不整合を検出または明示的に処理
T07	短距離・近接したポート	不可能な曲率を検出
T08	複数階層の PCell	トップセル座標へ正しく変換
T09	ルート作成後にインスタンス移動	古い接続情報を検出・再計算
T10	ネットリストを再抽出	幾何形状との整合性を確認

この表は**推奨テスト項目**であり、実行済みの試験結果を示すものではない。

15.10　推奨する共通データモデル（v5 以降の提案）

▼python

```
# 概念例：現行v4/v6.xのAPIではありません。
from dataclasses import dataclass

@dataclass(frozen=True)
class OpticalPort:
    instance_path: str
    name: str
    x_um: float
    y_um: float
    outward_deg: float
    width_um: float
    layer_id: str
```

方向を**外向き**に正規化して Router へ渡す方式を提案する。これにより、Pin 図形の方向規約と経路生成ロジックを切り離しやすくなる。ポート識別子には階層パスを

含める。

15.11　実装・公開前の確認事項

1. `photonics_pdk_v4.lym` の固定コミット ID を記録する。
2. Router v6.0/v6.2 の対象ファイルと固定コミット ID を記録する。
3. Pin Path と Text の正確な抽出規則をコードと照合する。
4. 回転・鏡映・階層配置でのポート変換をテストする。
5. 経路方式・曲率検証・障害物回避の実装範囲を版別に明記する。
6. 導波路長の算出方法と単位を検証する。
7. ネットリストの実際の出力形式を実データで確認する。

15.12　まとめ

PCell Library は**素子形状とポートの定義**、Photonics Router は**ポート間の接続経路の構築・評価**を担う。両者を安定して統合するには、ポートの座標系・方向規約・階層識別・曲率制約・長さ計算・ネットリスト整合性を共通仕様として定める必要がある。

次章では、PCell Library v5 に向けた共通 API、検証基盤、文書と実装のバージョン管理を検討する。

第 16 章

PCell Library v5 への発展 — 共通化・検証・Router 統合

対象: KLayout Silicon Photonics PDK v4 → v5（設計提案）**文書状態:** 技術解説ドラフト（2026-10-15）**関連章:** [第 3 章 アーキテクチャ](03_library_architecture.md)、[第 14 章 Optical Port](14_optical_ports.md)、[第 15 章 Router 連携](15_router_integration.md)

重要: 本章は v5 の**開発提案・ロードマップ**であり、v5 で実装済みの機能を記述したものではない。v4 の実装依存事項は、正式公開前に `photonics_pdk_v4.lym` の固定コミットと照合すること。

16.1 開発方針

v4 では Straight、Bend90、EulerBend、SBend、Ring、DirectionalCoupler、MMI_1x2、MZI、WaveguideRoute、PortConnector の 10 種類を対象に、幾何学的なレイアウトと光ポートを扱う。v5 では「PCell ごとに独立した生成処理」から「検証可能な共通基盤上の PCell 群」へ発展させる。

優先する品質目標は次の 5 点である。

1. **接続性:** ポートの位置、方向、幅、レイヤーが機械的に解釈できる。
2. **幾何学的健全性:** 無効パラメータ、自己交差、過小曲率半径を検出できる。
3. **再現性:** 同じ入力、同じ PDK 版から同じレイアウトを生成できる。
4. **相互運用性:** PCell と Photonics Router が共通のポート・経路データを使用する。
5. **検証可能性:** ユニットテスト、回帰テスト、サンプルレイアウトで変更を確認

できる。

16.2　提案するソフトウェア構成

▼text

```
photonics_pdk_v5/
 ├── __init__.py
 ├── library.py                # KLayout Library登録
 ├── technology.py             # レイヤー、DBU、設計ルール
 ├── geometry/
 │    ├── paths.py             # Path・Polygon変換
 │    ├── bezier.py            # Bé zier曲線
 │    ├── euler.py             # Euler曲線
 │    └── metrics.py           # 長さ・曲率・交差判定
 ├── ports/
 │    ├── model.py             # OpticalPortデータモデル
 │    ├── pin_io.py            # Pin Path/Textの読み書き
 │    └── transforms.py        # 回転・鏡映・階層変換
 ├── pcells/
 │    ├── straight.py
 │    ├── bend90.py
 │    ├── euler_bend.py
 │    ├── sbend.py
 │    ├── ring.py
 │    ├── directional_coupler.py
 │    ├── mmi_1x2.py
 │    ├── mzi.py
 │    ├── waveguide_route.py
 │    └── port_connector.py
 ├── integration/
 │    ├── router_adapter.py
 │    └── netlist_adapter.py
 └── tests/
      ├── test_geometry.py
      ├── test_ports.py
      ├── test_pcells.py
      └── test_router_integration.py
```

これは**推奨ディレクトリ構成**であり、既存リポジトリの実際の構成を表すものでは

ない。KLayout の.lym から Python モジュールを読み込む場合は、マクロ検索パスと配布方式を併せて設計する。

16.3 Optical Port API の統一

ポート情報を Pin 図形から推定するだけでなく、内部的には明示的なデータモデルを持つことを提案する。

▼python

```
# v5の概念設計例。現行v4のAPIではない。
from dataclasses import dataclass

@dataclass(frozen=True)
class OpticalPort:
    name: str
    x_um: float
    y_um: float
    angle_deg: float       # 内向き／外向きの規約を明示
    width_um: float
    layer: tuple[int, int]
    instance_path: str = ""
```

♣ 方向の規約

- 内向き・外向きのどちらを採用するか、API 単位で固定する。
- 既存の Pin 表現との変換関数を用意する。
- 角度は [0, 360) に正規化する。
- 鏡映を含む変換は、点座標だけでなく**方向ベクトル**に適用する。

♣ 一意なポート識別

opt1 という名前は複数の PCell に現れるため、接続管理では instance_path + port_name を一意な識別子として扱う。階層化・配列インスタンスの扱いもテストする。

16.4　共通幾何学エンジン

B é zier、Euler、円弧、直線を共通の「中心線」表現で扱い、各 PCell が必要に応じて導波路ポリゴンに変換する設計を提案する。

中心線を\(\mathbf r(t)=(x(t),y(t))\) とすると、長さは

$$L = \int_{t_0}^{t_1} \sqrt{x'(t)^2 + y'(t)^2}\, dt$$

曲率は

$$\kappa(t) = \frac{|x'(t)y''(t) - y'(t)x''(t)|}{(x'(t)^2 + y'(t)^2)^{3/2}}$$

で求められる。許容最小曲率半径を\(R_{\min}\) とすると、必要な条件は

$$\max_t \kappa(t) \le \frac{1}{R_{\min}}$$

である。ただし、数値サンプリングのみではサンプル点間の曲率極大を見落とす可能性がある。解析的な評価、適応的サンプリング、十分な誤差評価を組み合わせる。

重要: Path の線分長の合計は離散化された中心線長であり、元の連続曲線の厳密な弧長とは一致しない場合がある。長さ計算では両者を区別して記録する。

16.5　パラメータと設計ルールの検証

PCell の生成前に、少なくとも以下を検証する。

▼表 16.1: パラメータ・仕様一覧

項目	検証例	推奨処理
幅	`width > 0`	無効値は生成を拒否
長さ	`length > 0`	ゼロ・負値を拒否
分割数	曲線ごとの最小点数	範囲外は明示的なエラー
ギャップ	`gap >= 0` と製造ルール	違反を報告
曲率	`R_actual >= R_min`	警告または拒否
ポート	名前、幅、角度、位置	一貫性を検証
図形	自己交差・ゼロ面積	幾何検査を実施
DBU	量子化誤差	許容誤差を設定

製造可能性はプロセス依存である。最小線幅・最小間隔・曲げ半径などの値は、実際のファウンドリルールまたは使用技術に基づいて設定し、汎用的な固定値を「保証値」として扱わない。

16.6　10 種類の PCell 別改善候補

▼表 16.2: パラメータ・仕様一覧

PCell	v5 で検討する改善案
Straight	共通ポート生成、長さ検証、端点座標の保証
Bend90	曲率半径の定義統一、離散化精度の管理
EulerBend	最小曲率半径とフットプリントの明確化
SBend	端点接線・曲率制約、長さの高精度計算
Ring	結合部の幾何検証、バス／リングの属性管理
DirectionalCoupler	アクセス部の長さと結合部の定義整理
MMI_1x2	出力ポート配置と MMI 領域の幾何検証
MZI	アーム長差パラメータ、経路長比較、位相シフター拡張
WaveguideRoute	始終端の方向指定、曲率・障害物制約
PortConnector	接続補助の役割明確化、Straight との共通化

いずれも**開発候補**であり、現行 v4 の不足をソースコードからすべて実証した一覧ではない。

16.7　Photonics Router との共通インターフェース

推奨する処理フローは以下のとおり。

▼text

```
PCell生成 → ポート登録 → インスタンス変換
    → ポート接続要求 → 経路候補生成
    → 幾何検証 → 導波路配置 → 長さ・ネットリスト更新
```

接続要求には、接続元・接続先ポート ID、目標導波路幅、使用レイヤー、最小曲率半径、障害物条件を含める。Router の生成結果には、経路 ID、幾何形状、実測中心線長、検証結果を返す設計とする。

ネットリストは幾何学的な接続情報と区別する。 物理的に接触していても、幅不整合や向きの不整合などがあれば、検証済みの光学接続とは限らない。

16.8 テスト自動化と CI

♣ ユニットテスト

- 曲線の始点・終点と接線方向
- 直線・円弧の解析解との長さ比較
- 回転・鏡映後のポート位置と方向
- 無効パラメータに対するエラー処理

♣ 回帰テスト

- 10 種類の PCell を既定値で生成
- 代表的な境界パラメータで生成
- 図形数、ポート数、ポート名、境界ボックスを検査
- 必要に応じて GDS/OASIS 出力を比較

♣ 統合テスト

- Straight と Bend90 の接続
- MMI_1x2 と MZI の接続
- 回転・鏡映した PCell への Router 接続
- ネットリスト抽出後の接続数・ポート対応検査

CI 上で KLayout を実行する場合は、対象 OS・KLayout 版・Python 実行環境を固定し、GUI 不要のテストと GUI 依存テストを分離する。

16.9 段階的な開発ロードマップ

各 Phase の完了条件はプロジェクトの実装状況に応じて具体化する。

▼表 16.3: パラメータ・仕様一覧

段階	優先度	成果物	完了判定の例
Phase 0	最優先	v4 仕様の固定	Git コミット・パラメータ・ポート表を確定
Phase 1	高	共通 Port API	10 種類のポートを同じ API で取得
Phase 2	高	共通幾何・検証 API	長さ・曲率・DBU テストに合格
Phase 3	高	PCell 移行	既存 10 種類の回帰テストに合格
Phase 4	中	Router アダプタ	回転・鏡映を含む接続テストに合格
Phase 5	中	ネットリスト整合	レイアウトと接続情報の整合検査に合格
Phase 6	中	公開ドキュメント	使用例・変更履歴・移行ガイドを整備

16.10 v4 から v5 への互換性

既存設計を破壊しないために、次の方針を推奨する。

1. **PCell 名と主要パラメータ名を可能な限り維持**する。
2. 仕様変更が必要な場合は、旧名称のエイリアスまたは移行手順を提供する。
3. Optical Port の角度規約を変更する場合は、変換層を設ける。
4. v4 と v5 を別の Library 名で並行利用できるか検討する。
5. サンプル GDS と期待されるポート一覧を回帰基準として保存する。

既存 PCell の内部セル名・パラメータ型の変更は、保存済みレイアウトの再読込に影響する可能性がある。移行テストでは、**v4 で保存したレイアウトを v5 環境で開く**操作を必ず確認する。

16.11 公開前チェックリスト

- [ ] `photonics_pdk_v4.lym` の参照コミットを固定
- [ ] 第 1〜15 章の実装依存記述をソースコードと再照合
- [ ] 10 種類の PCell のパラメータ初期値を自動抽出して照合
- [ ] ポート位置・角度・レイヤーを実レイアウトで確認
- [ ] Router 各版の実装済み機能と提案機能を区別
- [ ] 数式の適用条件、近似条件、単位を明記
- [ ] 代表レイアウト画像とテスト結果を添付
- [ ] README の目次と相互リンクを検査
- [ ] ライセンス、著作権表記、変更履歴を確認

16.12　まとめ

v5 の開発では、新しい素子を追加する前に、**光ポートの意味を統一し、幾何形状を検証可能にし、Router と共通のデータモデルを共有すること**が重要である。

これにより、PCell 単体の作図ツールから、接続・長さ・ネットリストまで一貫して扱える Silicon Photonics レイアウト設計基盤へ発展させることができる。

関 連 資 料:　[技 術 解 説 書 の 目 次](README.md) / [第 14 章 Optical Port](14_optical_ports.md) / [第 15 章 Router 連携](15_router_integration.md)

付録 A
KLayout 実機検証記録

KLayout 0.30.9 で実行されたテスト結果（提供された実行ログ）を以下に記録します。

- PCell 初期生成・回転反転・パラメータ変更・GDS 往復: 71/71 PASS
- Optical Port の初期値・変更値・回転反転: 70/70 PASS
- 合計: 141/141 PASS

この結果はテストスクリプトで検査した範囲の合格を示します。DRC、光学シミュレーション、最小曲率半径保証、Router との統合検証は含みません。

オープンソース PDK で遊ぼう 9

KLayout でシリコンフォトニクス

2026 年 11 月 21 日　ver 1.0 （技術書典 21）

著　者　ほうた@ SIG
発行者　ほうた (SIG の遊び場)
連絡先　SIGnoAsobiba@gmail.com

© 2026 ほうた@ SIG

(powered by Re:VIEW Starter)

OPEN SOURCE PDK



SIG NO ASOBIBA